A PROJECT REPORT ON

"Cortex: An Agentic Retrieval-Augmented Generation System over Live Work Data with Explainable Memory and Consent-Gated Actions"

SAVITRIBAI PHULE PUNE UNIVERSITY

SUBMITTED TOWARDS THE
PARTIAL FULFILLMENT OF THE REQUIREMENTS OF

BACHELOR OF ENGINEERING
IN
COMPUTER ENGINEERING

BY

AJIT PARASKAR	BCOC17
HARSHVARDHAN PATIL	BCOC32
PRATHAMESH KUMBHAR	BCOC35
BIKRAM GHOSH	BCOC47

UNDER THE GUIDANCE OF
PROF. SWAPNALI GAIKWAD

DYP DPU
Dr. D. Y. Patil Institute of Technology, Pimpri, Pune

DEPARTMENT OF COMPUTER ENGINEERING

(2026-2027)

DYP DPU

Dr. D. Y. Patil Institute of Technology, Pimpri, Pune

CERTIFICATE

This is to certify that the Final Year Project report entitled **"Cortex: An Agentic Retrieval-Augmented Generation System over Live Work Data with Explainable Memory and Consent-Gated Actions"** submitted by **Ajit Paraskar** (BCOC17), **Harshvardhan Patil** (BCOC32), **Prathamesh Kumbhar** (BCOC35) and **Bikram Ghosh** (BCOC47) is a bonafide work carried out by them under the guidance of **Prof. Swapnali Gaikwad** in partial fulfillment of requirements for BE (Computer Engineering) of Savitribai Phule Pune University, Pune in the academic year 2026-27.

Prof. Swapnali Gaikwad
Guide
Department of Computer Engineering
Dr. DYPIT, Pune

Prof. (Dr.) Omkaresh Kulkarni
Head of Department
Department of Computer Engineering
Dr. DYPIT, Pune

External
Name and Signature

Prof. (Dr.) Nitin Sherje
Principal
Dr. DYPIT, Pune

Date: .....

Place: .....

DYP DPU

Dr. D. Y. Patil Institute of Technology, Pimpri, Pune

DECLARATION

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources.

We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea / data / fact / source in our submission.

We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Ajit Paraskar (BCOC17)
Harshvardhan Patil (BCOC32)
Prathamesh Kumbhar (BCOC35)
Bikram Ghosh (BCOC47) Date: _____

Acknowledgement

We take this opportunity to thank our project guide **Prof. Swapnali Gaikwad** for providing us the opportunity to work in an exciting and challenging field, agentic retrieval-augmented generation over live work data. Our discussions with our guide have been of immense help in defining the goals of this project and in identifying ways to achieve them.

Our honourable mention goes to **Prof. (Dr.) Nitin Sherje**, Principal of Dr. D. Y. Patil Institute of Technology, Pune, and **Prof. (Dr.) Omkaresh Kulkarni**, Head of the Computer Engineering Department, for granting us the opportunity to work as students, for providing us access to the laboratory and other research facilities that were of assistance to us in carrying out this project, and for their support and encouragement.

It is a great pleasure and immense satisfaction to express our deepest sense of gratitude and thanks to everyone who has directly or indirectly helped us in completing our project work successfully.

Ajit Paraskar
Harshvardhan Patil
Prathamesh Kumbhar
Bikram Ghosh

Abstract

Knowledge workers, developers and students keep their work in many separate tools: e-mail, calendars, shared drives, notes, team chat and code hosting. Answering "what changed in our project this week, and what is at risk?" means opening each tool and reconciling what they say. A typical Retrieval-Augmented Generation (RAG) chatbot runs one similarity search over a static collection, presents every sentence with equal confidence, silently picks one side when sources disagree, and either cannot act or acts without asking.

This project presents **Cortex**, an agentic RAG system over a user's *live* work data. For each question Cortex recalls what the user has said matters, plans two to five sub-questions, searches the relevant sources in parallel through a uniform connector layer (Gmail, Google Calendar, Google Drive, Notion, Slack and GitHub through Composio; any Model Context Protocol server; the web through Firecrawl), and streams an answer in which every factual sentence must cite numbered evidence. Each claim is then checked against the evidence it cites and a groundedness score is computed; unsupported claims are struck through, and contradictions between sources are shown with both citations. Requested actions are only *proposed* with an exact preview, and a separate approval endpoint is the only code path that executes them. Each memory records why it was kept, its source, salience and expiry, and the user can pin, edit or forget it. The system runs on free tiers through a failover router across several language model providers.

The project's research framing follows recent work on agentic RAG, in particular GRASP (GRanularity-Aware Search Policy for Agentic RAG), which studies when and how finely an agent should retrieve. In a preliminary 12-question evaluation over a fictional sample workspace, using free-tier models, Cortex reached a mean groundedness of 0.88 and a citation recall of 0.92, answered the unanswerable question honestly, executed no action without consent, and had a median latency of 3.7 s (p90 7.1 s). Conflict detection and action proposals each succeeded in one of two cases, which identifies the main directions for further work.

Contents

1 Synopsis **1**
- 1.1 Project Title . . . 2
- 1.2 Project Option . . . 2
- 1.3 Internal Guide . . . 2
- 1.4 Sponsorship and External Guide . . . 2
- 1.5 Technical Keywords (As per ACM Keywords) . . . 2
- 1.6 Problem Statement . . . 3
- 1.7 Abstract . . . 3
- 1.8 Goals and Objectives . . . 4
- 1.9 Relevant mathematics associated with the Project . . . 5
- 1.10 Names of Conferences / Journals where papers can be published 6
- 1.11 Review of Conference/Journal Papers supporting Project idea 6
- 1.12 Plan of Project Execution . . . 8

2 Technical Keywords **9**
- 2.1 Area of Project . . . 10
- 2.2 Technical Keywords . . . 10

3 Introduction **12**
- 3.1 Project Idea . . . 13
- 3.2 Motivation of the Project . . . 13
- 3.3 Literature Survey . . . 14

4 Problem Definition and scope **18**
- 4.1 Problem Statement . . . 19
  - 4.1.1 Goals and objectives . . . 19
  - 4.1.2 Statement of scope . . . 20
- 4.2 Major Constraints . . . 20
- 4.3 Methodologies of Problem solving and efficiency issues . . . 21
- 4.4 Outcome . . . 21
- 4.5 Applications . . . 23

4.6 Hardware Resources Required 23
4.7 Software Resources Required 23

5 Project Plan **26**
5.1 Project Estimates 27
5.1.1 Reconciled Estimates 27
5.1.2 Project Resources 27
5.2 Risk Management w.r.t. NP Hard analysis 28
5.2.1 Risk Identification 29
5.2.2 Risk Analysis 30
5.2.3 Overview of Risk Mitigation, Monitoring, Management 32
5.3 Project Schedule 32
5.3.1 Project task set 32
5.3.2 Task network 36
5.3.3 Timeline Chart 36
5.4 Team Organization 36
5.4.1 Team structure 36
5.4.2 Management reporting and communication 36

6 Software requirement specification **38**
6.1 Introduction 39
6.1.1 Purpose and Scope of Document 39
6.1.2 Overview of responsibilities of Developer 39
6.2 Usage Scenario 39
6.2.1 User profiles 39
6.2.2 Use-cases 40
6.2.3 Use Case View 40
6.3 Data Model and Description 40
6.3.1 Data Description 40
6.3.2 Data objects and Relationships 43
6.4 Functional Model and Description 43
6.4.1 Data Flow Diagram 44
6.4.2 Activity Diagram: 46
6.4.3 Non Functional Requirements: 46
6.4.4 State Diagram: 48
6.4.5 Design Constraints 48
6.4.6 Software Interface Description 49

7 Detailed Design Document using Appendix A and B **51**
7.1 Introduction 52
7.2 Architectural Design 52

7.3 Data design (using Appendices A and B) . . . 52
7.3.1 Internal software data structure . . . 54
7.3.2 Global data structure . . . 54
7.3.3 Temporary data structure . . . 55
7.3.4 Database description . . . 55
7.4 Component Design . . . 55
7.4.1 Class Diagram . . . 57
7.4.2 Interaction diagram for a consent-gated action . . . 57

8 Project Implementation 60
8.1 Introduction . . . 61
8.2 Tools and Technologies Used . . . 61
8.3 Methodologies/Algorithm Details . . . 61
8.3.1 Algorithm 1/Pseudo Code . . . 64
8.3.2 Algorithm 2/Pseudo Code . . . 65
8.3.3 Algorithm 3/Pseudo Code . . . 66
8.3.4 Algorithm 4/Pseudo Code . . . 67
8.4 Verification and Validation for Acceptance . . . 67

9 Software Testing 69
9.1 Type of Testing Used . . . 70
9.2 Test Cases and Test Results . . . 70

10 Results 74
10.1 Screen shots . . . 75
10.2 Outputs . . . 75

11 Deployment and Maintenance 81
11.1 Installation and un-installation . . . 82
11.2 User help . . . 83

12 Conclusion and future scope 84

13 References 87

Annexure A References 91

Annexure B Laboratory assignments on Project Analysis of Algorithmic Design 95

Annexure C Laboratory assignments on Project Quality and Reliability Testing of Project Design 99

Annexure D Project Planner **103**

Annexure E Reviewers Comments of Paper Submitted **105**

Annexure F Plagiarism Report **107**

Annexure G Term-II Project Laboratory Assignments **109**

Annexure H Information of Project Group Members **111**

List of Figures

5.1 Task network 36

6.1 Use case diagram. Dashed arrows are ≪include≫ relations. . 42
6.2 Data objects and relationships (ERD form) 44
6.3 Level 0 data flow diagram 44
6.4 Level 1 data flow diagram 45
6.5 Activity diagram of one agent run 47
6.6 State transition diagram 48

7.1 Architecture diagram 53
7.2 Class Diagram 58
7.3 Sequence diagram of a consent-gated action 59

10.1 Ask screen: a cited answer with a struck-through unsupported claim, a reported conflict between two sources, and the cited sources 75
10.2 Live trace: step timings, the three-step plan and each tool call with its query, result count and latency 76
10.3 A proposed Slack message waiting for approval, with an exact preview and Edit, Decline and Approve controls 76
10.4 Daily Brief: at-risk items, conflicts, upcoming deadlines and recent changes, each linked to its sources 77
10.5 Memory inspector (empty state) with the control to teach Cortex a memory of a chosen kind 78
10.6 Connections: the sample workspace with six sources, and the language model and memory status 78
10.7 Mobile layout 79
10.8 Groundedness per evaluation question 79
10.9 End-to-end time per evaluation question (Deep mode) 80

C.1 Venn diagram of answer sentences and their verification 100
C.2 Functional dependency graph 101

D.1 Timeline chart (Gantt chart) of the project 104

List of Tables

3.1 Comparison of related approaches with Cortex 17

4.1 Alternative methodologies and their efficiency 22
4.2 Hardware Requirements 24

5.1 Operating cost of the project 28
5.2 Risk Table 30
5.3 Risk Probability definitions [1] 31
5.4 Risk Impact definitions [1] 31
5.5 Team roles 37

6.1 Use Cases 41
6.2 Software interfaces 50

7.1 Database tables of the memory service 56

8.1 Tools and technologies 62
8.2 Acceptance checks 68

9.1 Test cases: model router (packages/llm) 71
9.2 Test cases: connectors (packages/connectors) 71
9.3 Test cases: agent, verification and actions (packages/agent) . . 72
9.4 Preliminary evaluation results (Deep mode, 12 questions) . . . 73

B.1 IDEA Matrix 96
B.2 IDEA matrix for Cortex 97
B.3 Knowledge canvas for Cortex 98

C.1 Black-box test cases for the identified functions 102

D.1 Project planner 104

CHAPTER 1

SYNOPSIS

1.1 Project Title

Cortex: An Agentic Retrieval-Augmented Generation System over Live Work Data with Explainable Memory and Consent-Gated Actions

1.2 Project Option

Internal project.

1.3 Internal Guide

Prof. Swapnali Gaikwad

1.4 Sponsorship and External Guide

The project is not sponsored, and there is no external guide.

1.5 Technical Keywords (As per ACM Keywords)

The keywords below follow the 1998 ACM Computing Classification System (`http://www.acm.org/about/class/ccs98-html`).

1. H. Information Systems
    (a) H.3 INFORMATION STORAGE AND RETRIEVAL
        i. H.3.3 Information Search and Retrieval
            A. Query formulation
            B. Retrieval models
            C. Search process
            D. Information filtering
        ii. H.3.4 Systems and Software
            A. Question-answering (fact retrieval) systems
            B. Distributed systems
            C. Performance evaluation (efficiency and effectiveness)
2. I. Computing Methodologies

(a) I.2 ARTIFICIAL INTELLIGENCE
    i. I.2.7 Natural Language Processing
        A. Language generation
        B. Language models
    ii. I.2.8 Problem Solving, Control Methods, and Search
        A. Plan execution, formation, and generation
    iii. I.2.11 Distributed Artificial Intelligence
        A. Intelligent agents

1.6 Problem Statement

To design and implement an agentic retrieval-augmented generation system that answers natural-language questions over a user's live work data, which is spread over e-mail, calendar, documents, notes, team chat, code hosting and the web. The system breaks each question into sub-questions and searches all relevant sources in parallel within fixed budgets. Every factual claim in its answer is cited and then checked against the evidence it cites. When two sources disagree, the system shows the contradiction instead of hiding it, and when information is missing it says so. It also keeps a long-term memory that the user can inspect and control, and it acts in the user's tools only after explicit consent for each action. All of this has to run on free-tier language model services.

1.7 Abstract

- The information a student or developer needs for daily work sits in Gmail, Google Calendar, Google Drive, Notion, Slack, GitHub and the web, and it changes every day. A typical RAG chatbot runs one similarity search over a fixed corpus and treats every retrieved sentence as equally reliable. If two sources disagree it quietly picks one, and it either cannot act at all or acts without asking. We built Cortex to close these gaps. For each question it plans two to five sub-questions, searches the connected sources in parallel through a registry of interchangeable connectors, recalls the priorities the user asked it to remember, and streams an answer in which every factual sentence cites numbered evidence. A verifier then checks each claim lexically: the citation must exist, the content words must overlap and the numbers must match. In Deep mode a model re-checks only the claims that look doubtful. The

user sees a groundedness score and a confidence state for each claim. Contradictions are shown with both citations, missing information is reported without spending a model call, and when a connector fails the answer says which one and what may be missing as a result. Actions appear as confirmation cards and run only through a single approval endpoint. A separate Python memory service records the provenance, salience decay and expiry of every memory and lets the user pin, edit or forget it; it uses Oracle Agent Memory when configured and SQLite otherwise. Operating cost stays at zero because a failover router moves between free-tier providers, with cooldowns and a response cache. In a preliminary 12-question evaluation the mean groundedness was 0.88 and the citation recall 0.92.

1.8 Goals and Objectives

- Answer multi-part questions over several live sources with a bounded plan of sub-questions and parallel retrieval.
- Cite every factual sentence and verify each claim against the evidence it cites; report a groundedness score.
- Surface cross-source contradictions as risks with both citations, and state missing information instead of guessing.
- Never execute an action in a user's tool without an explicit, single-use approval of an exact preview.
- Keep long-term memory selective and explainable (reason, source, salience, expiry), with pin, edit and forget controls.
- Degrade gracefully: name any source that fails, explain the impact, and continue with the rest.
- Keep the number of model calls per question small and predictable (at most five in Deep mode, at most three in Fast mode, counting the router call) so that the system runs on free tiers.
- Provide an evaluation harness and an automated test suite that measure groundedness, citation recall, honesty and consent safety.

1.9 Relevant mathematics associated with the Project

System Description: the system is modelled as $S = \{X, Y, C, M, P, F, Sc, Fc\}$, where C is the set of connectors, M the user's memory set and P the ordered list of language model providers.

- Input: $X = \{q, m, H, u\}$, where q is the question (a string of at most 2000 characters), $m \in \{\textit{fast}, \textit{deep}\}$ the run mode, H the last six turns of the conversation (each truncated to 1500 characters) and u the anonymous user identifier.

- Output: $Y = \{A, L, G, K, R, W\}$, where A is the answer in Markdown, L the list of claims with their status in $\{\textit{supported}, \textit{partial}, \textit{unsupported}\}$, $G \in [0, 1]$ the groundedness score, K the list of conflicts, R the list of action proposals (all in state *pending*) and W the set of memories written (at most two). The output is delivered as an ordered stream of typed events.

- Identify data structures, classes, divide and conquer strategies to exploit distributed/parallel/concurrent processing, constraints: the main data structures are the plan $\Pi = \{(q_i, S_i, k_i)\}_{i \le 5}$ of sub-questions q_i with source sets $S_i \subseteq C$ and keyword queries k_i; the evidence list E; least-recently-used caches with expiry; and the server-side map of pending actions. The question is divided into sub-questions, and the set of search calls $T = \bigcup_i \{(s, k_i) : s \in S_i\}$ (deduplicated, $|T| \le 18$) is executed concurrently, so the search latency is $\max_{t \in T} \tau_t$ rather than $\sum_{t \in T} \tau_t$, with each τ_t bounded by a 12 s timeout and one retry. Constraints: at most 5 sub-questions, 18 tool calls, 4 model calls in Deep mode (2 in Fast mode), 16 evidence items in Deep mode (8 in Fast mode), and a 90 s run timeout.

- Functions : Identify Objects, Morphisms, Overloading in functions, Functional relations: $F = \{f_{plan}, f_{search}, f_{rank}, f_{synth}, f_{verify}, f_{propose}, f_{remember}, f_{execute}\}$ with $f_{plan} : (q, C, M, H) \to \Pi$; $f_{search} : (s, k) \to E_{s,k}$; $f_{rank} : E \to E^*$; $f_{synth} : (q, \Pi, E^*, M) \to (A, K, R', W')$; $f_{verify} : (A, E^*) \to (L, G)$; $f_{propose} : R' \to R$; $f_{remember} : W' \to W$; and $f_{execute} : (r, \textit{approve}) \to \textit{result}$. The search function is overloaded (polymorphic) across five connector implementations behind one interface. The composition $f_{verify} \circ f_{synth} \circ f_{rank} \circ f_{search} \circ f_{plan}$ is the agent run; $f_{execute}$ is deliberately *not* part of that composition and can only be invoked by the user.

- Mathematical formulation if possible: evidence is ranked by $score(e) = 2\,h(e,q) + \max(0,\, 3 - a(e)/10)$, where h counts question keywords present in the evidence and $a(e)$ is its age in days (30 when unknown). A claim c citing evidence E_c has content-word overlap $o(c) = |T(c) \cap T(E_c)| \,/\, |T(c)|$, and is *supported* if $o(c) \geq 0.4$ and every number in c appears in E_c, *partial* if $o(c) \geq 0.2$, and *unsupported* otherwise. Groundedness is $G = \frac{1}{|L|} \sum_{c \in L} w(c)$ with $w = 1, 0.5, 0$ for the three states ($G = 1$ when L is empty). The effective salience of an unpinned memory of age t days is $s_{\mathit{eff}} = s \cdot 0.5^{\,t/30}$.

- Success Conditions: an answer is streamed in which every factual sentence carries a citation to existing evidence, G and the conflicts are reported, every proposal remains *pending* until the user approves it, and every unavailable source is named together with its impact.

- Failure Conditions: every language model provider is unavailable or rate-limited (the user is told so and offered Fast Mode); the 90 s run timeout expires; no source is connected; or the memory service is unreachable (the run continues without memory and says so).

1.10 Names of Conferences / Journals where papers can be published

- ACM SIGIR Conference on Research and Development in Information Retrieval (ACM).

- ACM IKDD CODS-COMAD (the ACM India data science conference, held in India) and workshops at IITs.

- Central Universities or SPPU Conferences (to be selected by the team with the guide).

- IEEE Access (IEEE journal) and the ACL Rolling Review venues (ACL, EMNLP, NAACL) for the verification and evaluation results.

1.11 Review of Conference/Journal Papers supporting Project idea

We read the papers below while shaping the idea. Each entry says in a line or two what the work did and what we took from it; Chapter 3 goes into

more detail.

1. **GRASP** [2] (base paper). Its agent picks one of three moves per turn: look something up by meaning, look it up by exact words, or open the paragraph around a sentence it already found. A reinforcement-learning reward teaches it when each move pays off. We did not train anything, but we copied the habit of starting from short quotes and capping how many retrieval turns a question gets.

2. **RAG** [3]. The original RAG model pairs a text generator with a dense retriever over a fixed Wikipedia index. Cortex keeps the core idea of answering from retrieved text, though our "index" is the user's live Gmail, Slack, GitHub and other accounts, queried at question time.

3. **ReAct** [4]. Here the model writes a thought, calls a tool, reads the result and repeats. Our planner also reasons before it searches, but the number of steps is decided up front so a run cannot loop.

4. **IRCoT** [5]. The authors fire a new retrieval after every reasoning sentence. Their results convinced us that one search round misses evidence for questions with several hops.

5. **Self-Ask** [6]. A model can answer each part of a question and still get the whole wrong; the paper calls this the compositionality gap and closes some of it by making the follow-up questions explicit. Cortex's planner writes its two to five sub-questions out in the same way.

6. **Self-RAG** [7]. One fine-tuned model decides for itself when to retrieve and grades its own output. In Cortex the grading is done outside the writer, by a separate verifier that looks at one claim at a time.

7. **Corrective RAG** [8]. A small judge rates what was retrieved and falls back to web search when the rating is poor. Our cheaper version is a single broadened retry for any sub-question that came back empty.

8. **Adaptive-RAG** [9]. A classifier guesses how hard a question is and picks no retrieval, one step or many steps. We hand that choice to the user as the Fast and Deep modes.

9. **ALCE** [10]. This benchmark scores how well generated answers cite their sources, and it is where our notion of citation recall comes from.

10. **RARR** [11]. RARR searches for support after the text is written and edits the parts it cannot back up. Cortex also checks after writing,

but it marks weak claims in the interface rather than silently rewriting them.

11. **FActScore** [12]. Long answers are broken into atomic facts and scored by the share that a knowledge source supports. Our groundedness score is a coarser cousin that works per sentence and only against the evidence that sentence cites.

12. **MemGPT** [13] and **Generative Agents** [14]. Both give an agent a long-term store and rank memories by how recent, important and relevant they are. Cortex ranks with similar signals, then shows the user why each memory exists and when it will lapse.

13. **Toolformer** [15] and the **MCP survey** [16]. The first teaches a model to call APIs; the second maps the Model Context Protocol ecosystem and its security holes. The second is the main reason our connectors may read freely but can only propose writes.

14. Other references: a survey of agentic RAG [17], the RAGAS metrics [18], the *Lost in the Middle* study [19], and two McKinsey reports on how knowledge workers spend their time and what generative AI may change [20, 21].

1.12 Plan of Project Execution

We planned the work as a waterfall, phase by phase, using the tasks in Section 5.3.1: survey and requirements first, then feasibility, design, coding, testing with evaluation, and finally the write-up. Figure 5.1 shows how the tasks depend on each other, and Annexure D holds the planner and timeline.

CHAPTER 2
TECHNICAL KEYWORDS

2.1 Area of Project

Information retrieval and natural language processing: agentic retrieval-augmented generation, multi-source question answering, claim verification (attribution), long-term memory for language-model agents, and tool use through the Model Context Protocol; implemented as a full-stack web application.

2.2 Technical Keywords

The keywords follow the 1998 ACM Computing Classification System (`http://www.acm.org/about/class/ccs98-html`).

1. H. Information Systems
    (a) H.3 INFORMATION STORAGE AND RETRIEVAL
        i. H.3.3 Information Search and Retrieval
            A. Query formulation
            B. Retrieval models
            C. Search process
            D. Information filtering
        ii. H.3.4 Systems and Software
            A. Question-answering (fact retrieval) systems
            B. Distributed systems
            C. Performance evaluation (efficiency and effectiveness)
2. I. Computing Methodologies
    (a) I.2 ARTIFICIAL INTELLIGENCE
        i. I.2.7 Natural Language Processing
            A. Language generation
            B. Language models
        ii. I.2.8 Problem Solving, Control Methods, and Search
            A. Plan execution, formation, and generation
        iii. I.2.11 Distributed Artificial Intelligence
            A. Intelligent agents

Additional free keywords: agentic RAG, query decomposition, parallel tool calling, claim verification, groundedness, citation, cross-source conflict detection, explainable memory, human-in-the-loop consent, Model Context Protocol, free-tier LLM failover.

CHAPTER 3
INTRODUCTION

3.1 Project Idea

- Cortex is a research assistant over the user's own, live tools. The user asks a question in plain language, for example "What changed in Orbit this week, and what's at risk?". Instead of a single similarity search, Cortex runs a short, bounded pipeline: *understand* (which sources are reachable now), *recall* (what the user said matters), *plan* (two to five sub-questions, each with its sources, keyword query and time window), *search* (every sub-question against every chosen source, in parallel), *synthesize* (a streamed answer that may use only numbered evidence and must cite it), *verify* (each claim checked against what it cites), *propose actions* (never execute) and *remember* (only salient, durable facts).
- The answer is presented as evidence rather than as prose to be trusted blindly. Each claim shows its verification state; unsupported claims are struck through; contradictions between sources (for example a Slack message and a GitHub milestone that give different dates) are listed as conflicts with both citations; and the full trace (plan, each tool call with its result count and latency, model calls) is available on demand.
- The same pipeline feeds a proactive *Daily Brief* (what changed in the last week and what is due in the next three, as cited cards), a *Memory* inspector and a *Connections* page for one-click OAuth through Composio.
- A fictional sample workspace (a four-person student team building a campus events app, with deliberate cross-source conflicts) uses the same connector interface and actions as the live connectors, so the system can be demonstrated and evaluated without any personal accounts.

3.2 Motivation of the Project

- **Fragmented information costs time.** In the words of the McKinsey Global Institute, "the average interaction worker spends an estimated 28 percent of the workweek managing e-mail and nearly 20 percent looking for internal information or tracking down colleagues who can help with specific tasks" [20]. Generative AI is expected to have its largest effect on exactly this kind of knowledge work [21].
- **One search is rarely enough.** A question like "who is blocking the release?" may need a Slack thread, then the GitHub issue it links to.

Work on multi-hop QA [5, 6] makes the same point. Dumping every retrieved chunk into the prompt does not fix it either, because models tend to overlook facts buried mid-way through a long prompt [19]. GRASP [2] treats "how much text to load" as a decision in its own right.

- **Trust needs evidence.** A model can write a confident sentence that nothing supports. Since citation quality [10] and fact-level support [12] can be measured, we decided Cortex should show the user, claim by claim, which parts are backed and which are not.

- **Agents that act must ask.** If an assistant can send an e-mail, then a misread request or an instruction hidden in a retrieved message can do real damage. MCP makes it easy to plug in tools, and that convenience also gives attackers more ways in [16]. Requiring a proposal and a separate approval is simple to build and easy to test.

- **Memory should be explainable.** Systems such as MemGPT and Generative Agents [13, 14] choose what to remember without asking. We wanted each memory in Cortex to carry its reason, its source and an expiry date, and to be deletable in one click.

- **Cost.** Our target users are students and solo developers who will not pay per-token prices, so Cortex had to run entirely on free tiers.

3.3 Literature Survey

- **Retrieval-augmented generation.** Lewis et al. [3] let a sequence-to-sequence model look things up in an external dense index instead of relying only on what its weights memorised. Their RAG-Sequence variant sums the answer probability over the k best passages z:

$$p(y \mid x) \approx \sum_{z \in \text{top-}k(p_\eta(\cdot \mid x))} p_\eta(z \mid x) \prod_i p_\theta(y_i \mid x, z, y_{1:i-1}).$$

  Retrieval happens once, before generation starts. We kept the principle of answering from retrieved text but dropped the single prebuilt index: Cortex searches several live tools through their own APIs and hands the model numbered quotes it must cite.

- **Agentic and multi-step retrieval.** In ReAct [4] the model writes a short thought, takes an action such as a Wikipedia lookup, reads

what came back, and goes round again; on HotpotQA and FEVER this produced fewer invented facts than reasoning alone. IRCoT [5] goes further and issues a fresh retrieval after every reasoning sentence, using that sentence as the query. Press et al. [6] noticed that models often get every sub-question right yet still botch the combined question. They named this the *compositionality gap* and reduced it by having the model pose its follow-up questions explicitly. The survey by Singh et al. [17] groups such systems under the label *agentic RAG*. Cortex fits the planning branch of that family: it writes one plan, then runs all of its searches at the same time.

- **Base paper: GRASP.** Gandhi, Lee et al. [2] point out that a retrieval agent keeps facing three choices at once: should it search now, should it match on exact words or on meaning, and how much surrounding text should it pull in, given that padding can distract it. Their agent has three moves (semantic search, keyword search, and expanding a hit to its paragraph), and its policy is learned with reinforcement learning. The reward mixes four things: a correct answer, reading that is grounded, using both search types where each helps, and finishing in few turns. On 500 validation questions each from HotpotQA, 2WikiMultiHopQA and MuSiQue they report retrieval recall of 0.90, 0.90 and 0.70, and better answers than every baseline they tried, from one-shot retrieval to prompted agents and other RL-trained agents. Looking at the learned behaviour, the authors see the agent search by meaning to land on the right topic, open a paragraph to check details, and then search by keyword for the exact name it found; they compare this to skimming and then scanning a text.

  We had no way to train such a policy on hosted free-tier models, so Cortex borrows three of GRASP's lessons as fixed rules instead. First, evidence is kept small: each item is a quote of a few hundred characters with title, author, time and link, and no more than 16 items reach the model. Second, searches use keywords, because the Gmail, Slack and GitHub search APIs are keyword-based anyway; only memory recall can be semantic, and only when Oracle Agent Memory is switched on. Third, the number of retrieval rounds is capped, with one wider retry for a sub-question that found nothing.

- **Adaptive and corrective retrieval.** Self-RAG [7] fine-tunes a single model to fetch passages only when it thinks it needs them and to emit special tokens that rate both the passage and its own answer. CRAG [8] puts a small grader in front of the generator; depending

on the grade, the retrieved documents are used, replaced by a web search, or combined with one, and are then trimmed to their useful strips. Adaptive-RAG [9] trains a classifier to guess question difficulty and picks no retrieval, one round or several rounds to match. Search-R1 [22] uses reinforcement learning to teach a model when to issue search queries in the middle of its reasoning. All four need training. Cortex gets a rougher form of the same adaptivity with none: the user picks Fast or Deep, the planner can label a message as *chat* and skip retrieval, and empty sub-questions get one retry.

- **Attribution and verification.** ALCE [10] is a benchmark for answers that carry citations. It scores fluency, correctness and citation quality, where citation recall checks that the cited passages fully back each statement and citation precision checks that no citation is superfluous. RARR [11] takes finished model output, searches for support, and edits away what it cannot support. FActScore [12] breaks text into atomic facts and reports

$$f(y) = \frac{1}{|A_y|} \sum_{a \in A_y} \mathbf{1}[a \text{ is supported by } \mathcal{C}],$$

that is, the share of atomic facts that a knowledge source $\mathcal{C}$ backs up. RAGAS [18] offers metrics for RAG pipelines that need no gold answers; one of them, faithfulness, asks whether the answer sticks to the retrieved context. Our groundedness score borrows from all of these but is cheaper: it works per sentence, compares each sentence only with the evidence it cites, uses word overlap first, and calls a model only for the sentences that look doubtful.

- **Long-term memory for agents.** MemGPT [13] borrows the idea of paging from operating systems: the prompt is treated as fast memory, and the agent swaps information in and out of a larger external store. In Generative Agents [14], simulated characters log everything they observe and later pull memories back by a weighted score of recency, importance and relevance. Cortex's memory service ranks with the same three ingredients (salience for importance, a 30-day half-life for recency, search score for relevance). The difference is that the user can see them: every memory lists why it was saved and where it came from, and it can be pinned, edited or forgotten from the Memory page.

- **Tool use and the Model Context Protocol.** Toolformer [15] lets a model teach itself, from its own sampled API calls, which tool to

call and with what arguments. MCP [23] is an open protocol that gives assistants one common way to reach tools and data. Hou et al. [16] map the MCP ecosystem and list the attacks possible at each stage of a server's life, from creation to updates. Cortex talks MCP in two places, Composio's hosted endpoint and any extra server listed in `mcp.config.json`, and in both cases a write is only ever a proposal waiting for the user.

- **Long contexts.** Liu et al. [19] moved the relevant passage around inside long prompts and found a U-shaped curve: models do best when the fact is near the start or the end, and worst when it sits in the middle. That result, together with GRASP's point about granularity, is why Cortex sends a short, ranked list of evidence rather than everything it found.

Table 3.1 compares the closest approaches with Cortex on the properties this project targets.

Approach	Multi-step	Training-free	Claim check	Live tools	Consent gate
RAG [3]	No	No	No	No	–
ReAct [4]	Yes	Yes	No	Yes	No
IRCoT [5]	Yes	Yes	No	No	–
Self-RAG [7]	Yes	No	Self-critique	No	–
CRAG [8]	Partly	No	Retrieval check	Web	–
Adaptive-RAG [9]	Adaptive	No	No	No	–
GRASP [2]	Yes	No (RL)	Grounded-reading reward	No	–
Cortex (this work)	Yes (bounded)	Yes	Per claim	Yes	Yes

Table 3.1: Comparison of related approaches with Cortex

CHAPTER 4

PROBLEM DEFINITION AND SCOPE

4.1 Problem Statement

A user's work data lives in several tools, each hosted separately. Given a question q from this user, Cortex has a fixed allowance of model calls and tool calls in which to produce an answer. That answer may use only evidence fetched from the tools during the run (a), and every factual sentence in it must cite its evidence (b). It must also say how much of the answer is supported (c), where the sources disagree and what could not be found (d). If the user asks for a write operation, such as sending a message, the system turns it into a proposal that runs only after the user approves it (e). Sources and model providers will sometimes fail, and the system has to keep working when they do. Finally, it should remember only what is likely to matter later, and the user must be able to see and change what it remembers.

4.1.1 Goals and objectives

Goal and Objectives:

- **Overall goal:** a trustworthy, free-to-run research assistant over live work data.
- **Input (ask):** an HTTP `POST /api/ask` request with a JSON body {`question, mode, threadId?, history?`}. `question` is a non-empty string of at most 2000 characters; `mode` is `"fast"` or `"deep"` (any other value is treated as `"deep"`); `history` is the recent conversation, of which the last six turns are used, each cut to 1500 characters. The user identity is an anonymous per-browser identifier held in an HTTP-only cookie.
- **Output (ask):** a `text/event-stream` response. Each event is one JSON object of a fixed type: `run`, `step` (seven steps, each `active`, `done` or `skipped`, with its duration), `memory_used`, `plan`, `tool_call`, `tool_result`, `evidence`, `source_error`, `token`, `answer`, `verdict` (claims, groundedness, conflicts, missing items, follow-up questions), `action_proposal`, `memory_written`, `usage`, `error` and `done`.
- **Input/output (approve):** `POST /api/actions/{id}` with {`decision: "approve" | "reject", edits?`} returns the updated proposal with status `executed`, `failed` or `rejected`; a second decision on the same proposal is refused with HTTP 409.

- **Other outputs:** the Daily Brief (`GET /api/brief`), the memory list and its pin, edit and forget operations (`/api/memory`), and the connection list and OAuth start (`/api/connections`).

4.1.2 Statement of scope

- **Size and bounds of input:** question length ≤ 2000 characters (longer questions are rejected with a message); history ≤ 6 turns; plan ≤ 5 sub-questions; search calls ≤ 18 per run; each connector returns at most 6 items per call (4 in Fast mode); at most 16 evidence items (8 in Fast mode) reach the model; at most 3 action proposals and 2 memory writes per run. **Input validation:** an empty question is rejected; planner output is validated against a schema and replaced by a local single-step plan if unusable; action proposals are validated against the connector's declared actions and required parameters; memory writes are validated for length (3–500 characters) and salience in $[0, 1]$. **Input dependency:** the plan depends on the connected sources and on recalled memories; the answer depends on the evidence; verification depends on the answer and the evidence. The I/O state behaviour of an action is shown in Figure 6.6.

- **What the product is:** a web application and a set of reusable packages (agent, connectors, model router) plus a memory service, which answer questions over connected tools with cited, verified answers, propose actions, and keep an explainable memory. **What it is not:** it is not a search index or a data warehouse (it does not copy users' data; it queries the tools at question time and caches results for 90 s), it does not train or fine-tune models, it does not execute any write operation on its own, and it does not yet implement the time-travel (snapshot diff) queries, the editable plan, or the Cortex-as-an-MCP-server interface described in the project vision; those are future work.

4.2 Major Constraints

- **Free-tier quotas.** All language models are used through free tiers with per-minute and per-day limits that change often. This constrains the number of model calls per question (at most 5 in Deep mode, 3 in Fast mode), requires failover across providers with cooldowns, and motivates the response cache.

- **Third-party APIs.** Connectors depend on Composio, the MCP servers and Firecrawl; tool names and output shapes can change. Tool identifiers are therefore stored as data, and tool output is mapped to evidence by a tolerant extractor.
- **Privacy and consent.** The system reads private mail and chat. It must not store users' source data beyond short-lived caches, must not expose OAuth tokens to the browser, and must not perform write operations without explicit approval.
- **Latency.** An interactive answer should start within a few seconds. Searches run in parallel with a 12 s timeout per call, and the answer is streamed.
- **Memory backend.** Oracle Agent Memory provides only a Python SDK, which forces a separate Python service. The Oracle AI Database container could not be used at the time of writing because of a Docker networking issue, so the SQLite fallback is the active backend.
- **Team and time.** A four-member team within one academic year, alongside regular coursework.

4.3 Methodologies of Problem solving and efficiency issues

- The problem can be solved in several ways. Table 4.1 compares them on the number of model calls per question, which dominates both latency and free-tier cost, and on other efficiency and quality issues.

We added a few more savings to the chosen design. A prompt that has been sent before is answered from a least-recently-used cache, which keeps entries for 6 h by default and is saved to disk. The same search repeated within 90 s comes from a separate search cache. Only the synthesis step uses the stronger model tier; the planner and the verifier make do with the faster one. The lexical verifier is linear in the number of claim and evidence tokens, so we run it first and call a model only for what it cannot settle.

4.4 Outcome

- A working web application (Ask with live trace, Brief, Memory, Connections) and an agent API that answers questions over six work tools

Approach	Model calls	Efficiency and quality issues
Single-shot RAG over a copied index	1	Needs ingesting and re-indexing every tool; stale data; one retrieval round misses multi-hop evidence; no claim check.
Unbounded ReAct loop	n (unbounded)	Calls grow with the number of steps; hard to keep within free quotas; latency varies widely; can loop.
RL-trained search policy (GRASP, Search-R1)	several per question	Best retrieval decisions, but requires training data, GPUs and a model the team can fine-tune; not feasible with hosted free-tier APIs.
Graph framework with human interrupts	varies	Flexible, but adds a framework dependency and makes the call count less predictable.
Bounded plan-then-execute pipeline (chosen)	Deep ≤ 5, Fast ≤ 3	One planning call, parallel searches (no model call), one streamed synthesis, an optional verification call for doubtful claims only. "Not found" answers use no model call. Predictable cost and latency.

Table 4.1: Alternative methodologies and their efficiency

and the web with cited, verified answers.

- Reusable packages. The model router handles free-tier failover, cooldowns and caching. The connector registry ships with Composio, generic MCP, Firecrawl and sample-workspace connectors. The memory service tracks provenance and decay and lets the user govern what is kept.
- A consent model in which only one endpoint can execute an action, once, for the user who received the proposal, within 30 minutes.
- An automated test suite (18 tests) and an evaluation harness with preliminary results (mean groundedness 0.88, citation recall 0.92, zero actions executed without consent).

4.5 Applications

- A student project team can ask what the guide asked for, which deadline moved and who is stuck, without opening mail, chat, Drive and GitHub one by one.
- A developer can check which pull requests wait on them and whether the release date in Slack matches the GitHub milestone.
- Anyone who starts the day with e-mail can read the Daily Brief instead, ordered by the priorities Cortex has been told to remember.
- A team that needs to audit an assistant can follow each claim back to its source, and nothing is sent or changed without a person approving it.

4.6 Hardware Resources Required

4.7 Software Resources Required

Platform :

1. Operating System: Windows 10/11, Linux or macOS (development was done on Windows 11).
2. IDE: Visual Studio Code.

Sr. No.	Parameter	Minimum Requirement	Justification
1	CPU Speed	2 GHz, dual core	Runs the Next.js server, the Python memory service and the browser at the same time; model inference happens at the providers.
2	RAM	8 GB	The Next.js development server, Node.js, the Python service and a browser together need several gigabytes; the optional Oracle Database Free container needs more.
3	Storage	5 GB free	Node.js dependencies, the Python virtual environment, caches and the SQLite ledger.
4	Network	Broadband Internet	Every model call and live connector call is a remote API request.

Table 4.2: Hardware Requirements

3. Programming Language: TypeScript on Node.js 24 or later (web application, agent, connectors, router); Python 3.11 (memory service).

4. Frameworks and libraries: Next.js 16 with React 19 and Tailwind CSS 4; Zod; the OpenAI-compatible client library; the Composio SDK; the MCP TypeScript SDK; the Firecrawl SDK; the LangSmith SDK; FastAPI, Uvicorn and Pydantic; `oracledb` and `oracleagentmemory` (optional); Vitest.

5. External services (free tiers): Groq, Google Gemini, Cerebras, OpenRouter and Puter (model providers); Composio; Firecrawl; LangSmith; Oracle AI Database (optional).

CHAPTER 5
PROJECT PLAN

5.1 Project Estimates

The department's format asks for a waterfall plan, so we followed one. Requirements, feasibility, design, coding, testing and maintenance each had to be signed off before the next phase began. The estimates below use the basic COCOMO model together with the outputs of the laboratory assignments in Annexures B and C.

5.1.1 Reconciled Estimates

5.1.1.1 Cost Estimate

The size of the delivered source code, counted as non-blank lines of TypeScript, TSX, Python, CSS and scripts (excluding dependencies and generated files), is about 6.1 KLOC. The basic COCOMO model for an organic project gives

$$E = 2.4 \times (\text{KLOC})^{1.05} = 2.4 \times 6.1^{1.05} \approx 16.0 \text{ person-months.}$$

The monetary cost of operation is zero: every external service is used on its free tier (Table 5.1). The only expenses are the team's own computers and Internet connection. If a notional labour rate is needed for comparison, the effort can be multiplied by a rate chosen by the department; no such rate is assumed here.

5.1.1.2 Time Estimates

The development time from basic COCOMO is

$$D = 2.5 \times E^{0.38} = 2.5 \times 16.0^{0.38} \approx 7.2 \text{ months,}$$

with an average staffing of $E/D \approx 2.2$ persons. With four team members working part-time alongside coursework, this fits the academic year (about ten months including documentation and evaluation). The phase-wise schedule is in Section 5.3 and Annexure D.

5.1.2 Project Resources

- **People:** four students (see Section 5.4) and the internal guide.
- **Hardware:** the team's laptops (Table 4.2); no dedicated server is needed for development.

Resource	Plan used	Cost (INR)
Language models (Groq, Gemini, Cerebras, OpenRouter)	Free tier	0
Puter (last-resort model fallback)	User-pays free credit	0
Composio (managed OAuth connectors)	Free plan	0
Firecrawl (web evidence)	Free plan	0
LangSmith (tracing)	Developer (free) plan	0
Oracle AI Database	Free container / Always Free	0
Development tools (VS Code, Node.js, Python, Git)	Open source / free	0

Table 5.1: Operating cost of the project

- **Software and tools:** as listed in the software resources section; Git for version control.
- **Memory sharing, IPC and concurrency:** the web server and the memory service run as two processes and talk over HTTP/JSON on the same machine. The browser receives the agent's progress as a server-sent event stream. Inside the server, search calls run at the same time as asynchronous promises, while the memory service puts a lock around its SQLite database so that writes do not collide. A few objects exist once per Node.js process (the model router, the action store and the connector registries), and every request shares them.

5.2 Risk Management w.r.t. NP Hard analysis

Here we list what could derail Cortex and what we did about each item.

5.2.1 Risk Identification

We went back over our scope, the requirements and the schedule, and answered the risk questionnaire from Pressman and Maxim [1]. The questions below are quoted from that book, with our answer in italics after each one. The risks it surfaced are sorted into Pressman's categories and collected in Table 5.2.

1. Have top software and customer managers formally committed to support the project? *Yes; it is an internal project approved by the department, with no external customer.*

2. Are end-users enthusiastically committed to the project and the system/product to be built? *Not yet established; there are no real users yet, so the sample workspace and the evaluation stand in for user feedback.*

3. Are requirements fully understood by the software engineering team and its customers? *Mostly; the core requirements are fixed, while some vision items (time-travel queries, editable plan) are deferred.*

4. Have customers been involved fully in the definition of requirements? *No external customer; requirements come from the team and the guide.*

5. Do end-users have realistic expectations? *The interface states uncertainty explicitly (struck-through claims, "not found" answers) to keep expectations realistic.*

6. Does the software engineering team have the right mix of skills? *Partly; agent design, web development and Python are covered, while database operations (Oracle) were new.*

7. Are project requirements stable? *The functional requirements are stable; external APIs and free-tier limits are not.*

8. Is the number of people on the project team adequate to do the job? *Yes for the current scope (Section 5.4).*

9. Do all customer/user constituencies agree on the importance of the project and on the requirements for the system/product to be built? *Yes within the team and with the guide.*

5.2.2 Risk Analysis

Table 5.2 rates each risk by how likely it is and how much it would hurt the schedule and the quality of answers. Tables 5.3 and 5.4 give the scales.

ID	Risk Description	Probability	Impact		
			Schedule	Quality	Overall
1	Free-tier model limits, quota changes or model removals stop answers	High	Medium	High	High
2	Connector APIs (Composio tool names, MCP servers) change or fail	Medium	Medium	High	High
3	Answers contain unsupported or hallucinated claims	Medium	Low	Very High	High
4	The agent performs an unintended action (e.g. after prompt injection)	Low	Low	Very High	Very High
5	The Oracle memory backend cannot be brought up	High	Medium	Medium	Medium
6	Private user data is exposed or retained	Low	Low	Very High	High

Table 5.2: Risk Table

Probability	Value	Description
High	Probability of occurrence is	$> 75\%$
Medium	Probability of occurrence is	$26 - 75\%$
Low	Probability of occurrence is	$< 25\%$

Table 5.3: Risk Probability definitions [1]

Impact	Value	Description
Very high	$> 10\%$	Schedule impact or Unacceptable quality
High	$5 - 10\%$	Schedule impact or Some parts of the project have low quality
Medium	$< 5\%$	Schedule impact or Barely noticeable degradation in quality Low Impact on schedule or Quality can be incorporated

Table 5.4: Risk Impact definitions [1]

5.2.3 Overview of Risk Mitigation, Monitoring, Management

Each risk from Table 5.2 is expanded below with its source, our response and its current status.

Risk ID	1
Risk Description	Free-tier model limits, quota changes or model removals stop answers.
Category	Technology
Source	Live testing with real keys (rate-limit and "model not found" responses).
Probability	High
Impact	High
Response	Mitigate
Strategy	Failover router across several providers; cooldowns per error type (rate limit, invalid key, out of credit, model not found); one wait of up to 12 s for the soonest provider; response cache; model identifiers configurable through environment variables; Fast Mode as a one-call fallback.
Risk Status	Occurred; mitigated

5.3 Project Schedule

5.3.1 Project task set

We split the year into ten tasks:

- Task 1: Problem identification, literature survey (base paper GRASP and related work) and requirement gathering.
- Task 2: Software requirement specification, feasibility study and risk analysis.

Risk ID	2
Risk Description	Connector APIs change or fail.
Category	Technology / External dependency
Source	Software Design Specification documentation review.
Probability	Medium
Impact	High
Response	Mitigate
Strategy	Tool identifiers kept as data in one file; tolerant evidence extraction from any output shape; per-call timeout, one retry and a short cache; a failed source is reported with its impact while the run continues; a service check script verifies every configured tool.
Risk Status	Identified; mitigated

Risk ID	3
Risk Description	Answers contain unsupported or hallucinated claims.
Category	Product quality
Source	Literature survey and early testing.
Probability	Medium
Impact	Very High
Response	Mitigate
Strategy	Evidence-only prompt that requires a citation per factual sentence; lexical claim verification with a model recheck for doubtful claims; struck-through unsupported claims and a groundedness score in the interface; “not found” answered without a model call; groundedness measured by the evaluation harness.
Risk Status	Identified; monitored by evaluation

Risk ID	4
Risk Description	The agent performs an unintended action.
Category	Security
Source	Threat review of tool use and MCP [16].
Probability	Low
Impact	Very High
Response	Avoid
Strategy	The agent can only propose; proposals are validated, stored server-side and shown with an exact preview; one endpoint executes an approved proposal once, for the same user, within 30 minutes; tests and the evaluation check that nothing runs without consent.
Risk Status	Identified; controlled (0 unconsented executions measured)

Risk ID	5
Risk Description	The Oracle memory backend cannot be brought up.
Category	Development Environment
Source	Setting up the local Oracle AI Database Free container.
Probability	High
Impact	Medium
Response	Mitigate
Strategy	The memory service offers the same API on SQLite with full-text search and reports the active backend on its health endpoint; Oracle remains optional.
Risk Status	Occurred (Docker networking issue); fallback in use

Risk ID	6
Risk Description	Private user data is exposed or retained.
Category	Security / Privacy
Source	Requirements review.
Probability	Low
Impact	Very High
Response	Mitigate
Strategy	Managed OAuth so the browser never sees tokens; no copying of source data into a store (only 90 s search caches); only selected memories are kept, each with an expiry and a forget control; memories of the sample workspace are kept apart from real ones.
Risk Status	Identified

- Task 3: Architecture and detailed design (agent pipeline, connectors, router, memory, interfaces).
- Task 4: Feasibility spikes: free-tier model failover, Composio OAuth and tools, Oracle Agent Memory.
- Task 5: Implementation of the core: model router, connector registry and connectors, agent pipeline, action store, memory service.
- Task 6: Implementation of the web application: Ask with live trace, Brief, Memory, Connections.
- Task 7: Testing and evaluation: unit and integration tests, evaluation harness, live service checks.
- Task 8: Term-I report and paper preparation.
- Task 9: Term-II work: Oracle memory backend, larger evaluation set, deployment.
- Task 10: Final report, demonstration and submission.

5.3.2 Task network

Figure 5.1 shows which task waits on which. The feasibility spikes (T4) ran beside the design so that the riskiest services were tried before coding began.

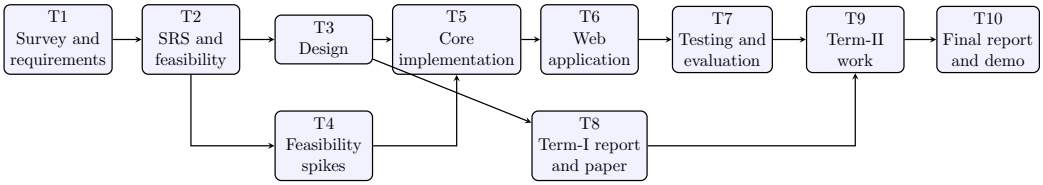


Figure 5.1: Task network

5.3.3 Timeline Chart

The month-by-month timeline is in the planner in Annexure D, drawn as a Gantt chart in Figure D.1.

5.4 Team Organization

The team consists of four students supervised by the internal guide. Work is divided by subsystem, and each subsystem has one owner and one reviewer.

5.4.1 Team structure

Table 5.5 lists the roles. The assignment of members to roles is to be completed by the team.

5.4.2 Management reporting and communication

We report to the guide in the project slots of the lab timetable, and progress is noted on the assessment sheet. Day to day, the Git repository is our record: every change lands as a commit with a message saying what and why, design decisions go into the docs folder, and nothing is merged until the tests and the evaluation have been run.

Role	Responsibilities	Member
Agent and verification	Planner, parallel search, synthesis prompt, claim verification, action store, evaluation harness	[to be filled]
Connectors and model router	Composio, MCP, Firecrawl and sample connectors; free-tier router, cooldowns and cache	[to be filled]
Memory service	FastAPI service, SQLite ledger, Oracle Agent Memory adapter, memory policy	[to be filled]
Web application and testing	Ask, Brief, Memory and Connections screens; event stream client; tests and documentation	[to be filled]
Internal guide	Review of design and progress, approval of milestones	Prof. Swapnali Gaikwad

Table 5.5: Team roles

CHAPTER 6

SOFTWARE REQUIREMENT SPECIFICATION

6.1 Introduction

6.1.1 Purpose and Scope of Document

This chapter pins down what Cortex has to do and under which limits. It starts with who uses the system and why, moves on to the data it keeps and how a question flows through it, and ends with the non-functional requirements, design constraints and interfaces. We wrote it for ourselves, our guide and the examiners; the design in Chapter 7 and the tests in Chapter 9 both trace back to it.

6.1.2 Overview of responsibilities of Developer

- Gather and document requirements; study the base paper and related literature.
- Design the agent pipeline, the connector interface, the model router, the memory policy and the consent model.
- Implement the web application, the packages and the memory service; keep secrets on the server only.
- Write automated tests and an evaluation harness, run them before each change, and report results honestly.
- Document installation, configuration and use; maintain the system as external APIs and free-tier limits change.

6.2 Usage Scenario

Below are the people and systems that interact with Cortex, followed by what each of them does with it.

6.2.1 User profiles

- **End user** (primary actor): typically a developer, an undergraduate or anyone else whose work is split across Gmail, Google Calendar, Google Drive, Notion, Slack and GitHub. We assume this person may not be technical, so they never deal with API keys or configuration files. They ask questions and read the Brief. When Cortex proposes an action they approve or decline it, and they look after their own memories.

- **Operator** (deploys Cortex; may be the same person): places API keys in the server's environment file, optionally declares extra MCP servers in a configuration file, and starts the web server and the memory service.
- **Connected applications** (secondary actor): the six apps above, reached through Composio, plus any MCP server and the web (through Firecrawl). They answer searches, and they carry out actions once the user has approved them.
- **Language model providers** (secondary actor): Groq, Google Gemini, Cerebras, OpenRouter and Puter, reached through OpenAI-compatible endpoints.
- **Memory service** (secondary actor): the Python service that stores and searches memories, backed by SQLite or Oracle AI Database.

6.2.2 Use-cases

Table 6.1 lists the eight use cases.

6.2.3 Use Case View

Figure 6.1 draws the same use cases, with the user on the left and the supporting systems on the right.

6.3 Data Model and Description

6.3.1 Data Description

Cortex does not copy users' source data into a database: evidence exists only during a run (and in a 90 s search cache). The persistent data are the memories and the conversation messages kept by the memory service, and the response cache of the model router. The main data objects are:

- **Evidence**: one retrieved item, with a run-scoped identifier (`e1`, `e2`, ...), source, title, quote (a compact excerpt), and optional link, author, timestamp and sub-question identifier. Evidence is numbered in the prompt and cited as `[n]`.

Sr No.	Use Case	Description	Actors	Assumptions
1	Ask a question	Cortex plans sub-questions, searches sources in parallel, streams a cited answer and verifies each claim	User, connected apps, model providers	At least one source connected and one provider key configured
2	Choose Fast or Deep mode	Fast: local single-step plan and one model call; Deep: model plan, empty-result retry and model re-check of doubtful claims	User	–
3	Inspect trace and evidence	Open citations, cited sources, the plan, each tool call with its count and latency, and model usage	User	A run has started
4	Approve, edit or decline an action	Review the exact preview of a proposed action, optionally edit it, then approve (executes once) or decline	User, connected app	Proposal is pending, less than 30 minutes old, same user
5	Read the Daily Brief	Cited cards for what changed, what is at risk, conflicts and what is upcoming	User, connected apps, model providers	Sources connected
6	Manage memory	Teach a memory, pin, edit or forget it, and see why it was kept	User, memory service	Memory service running
7	Connect an app or choose a workspace	Start OAuth through Composio, or switch between the sample workspace and the user's own accounts	User, Composio	Composio key configured for own accounts
8	Add an MCP	Declare a server	Operator	Server reachable

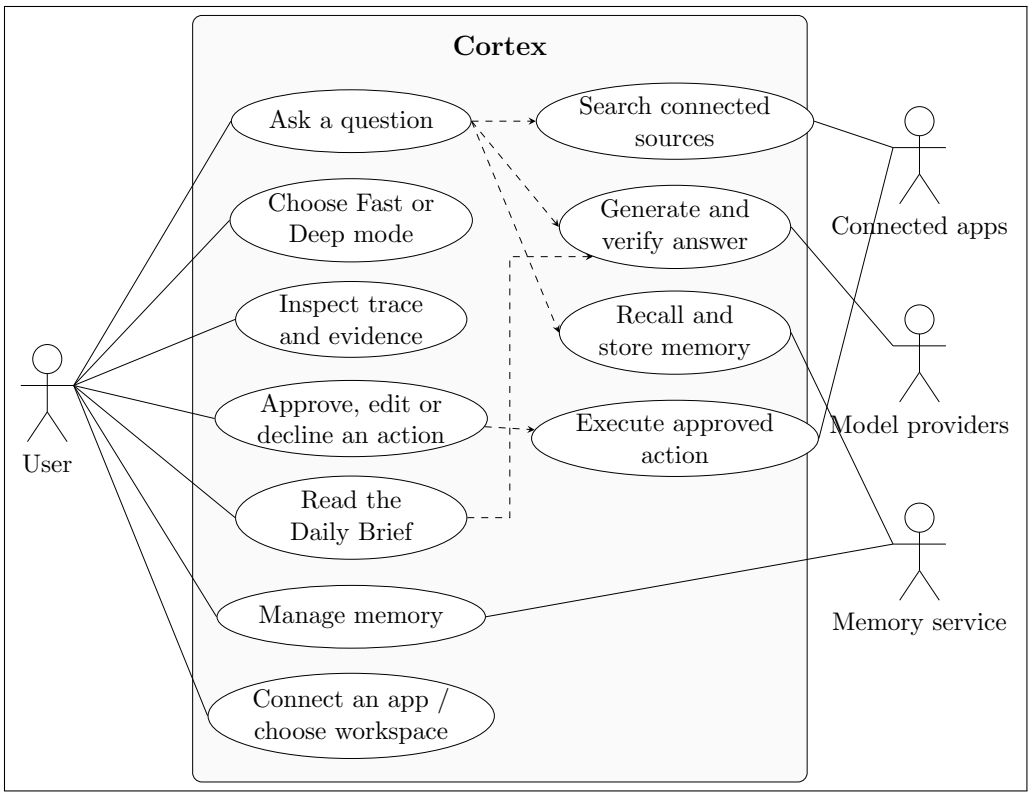


Figure 6.1: Use case diagram. Dashed arrows are ≪include≫ relations.

- **Plan** and **SubQuestion**: the run intent (`research`, `action` or `chat`), the rewritten question, an optional time window (since, until, label), and up to five sub-questions, each with its sources and a keyword query of at most 80 characters.
- **Claim**: a sentence or list item of the answer, the identifiers of the evidence it cites, and its status (`supported`, `partial`, `unsupported`).
- **Conflict**: a summary of a disagreement between sources and the identifiers of the evidence on each side.
- **ActionProposal**: identifier, connector, action, title, a human-readable preview, arguments, risk (`low` or `high`), status (`pending`, `approved`, `rejected`, `executed`, `failed`) and result.
- **MemoryItem**: identifier, user, content, kind (`priority`, `preference`, `fact`, `person`, `decision`), source, reason, salience in $[0, 1]$, pinned flag, creation time, expiry time and the run that created it.
- **SourceFailure**: the source, its display name, the reason (for example "its connection expired; reconnect it") and the impact on the answer.
- **AgentEvent**: the unit of the output stream; one of sixteen typed events listed in Section 4.1.1.

6.3.2 Data objects and Relationships

Figure 6.2 draws these objects as an entity-relationship diagram. Each run has exactly one plan, and the plan holds up to five sub-questions, each of which yields its own evidence. A claim may cite any number of evidence items, including none, whereas a conflict always points to evidence from at least two sides. A run can also produce action proposals and memories. Its messages are appended to the conversation thread it belongs to.

6.4 Functional Model and Description

This section follows a question through Cortex as a flow of data. The classes that implement each step are left for Chapter 7. Nine functions take part in a run: (1) check which sources are available; (2) recall memories; (3) plan; (4) search in parallel and rank evidence; (5) synthesize a streamed, cited answer; (6) verify claims and compute groundedness; (7) validate and store action proposals; (8) remember salient facts; and (9) execute an approved action. The Daily Brief reuses functions (1), (2), (4) and a single synthesis call.

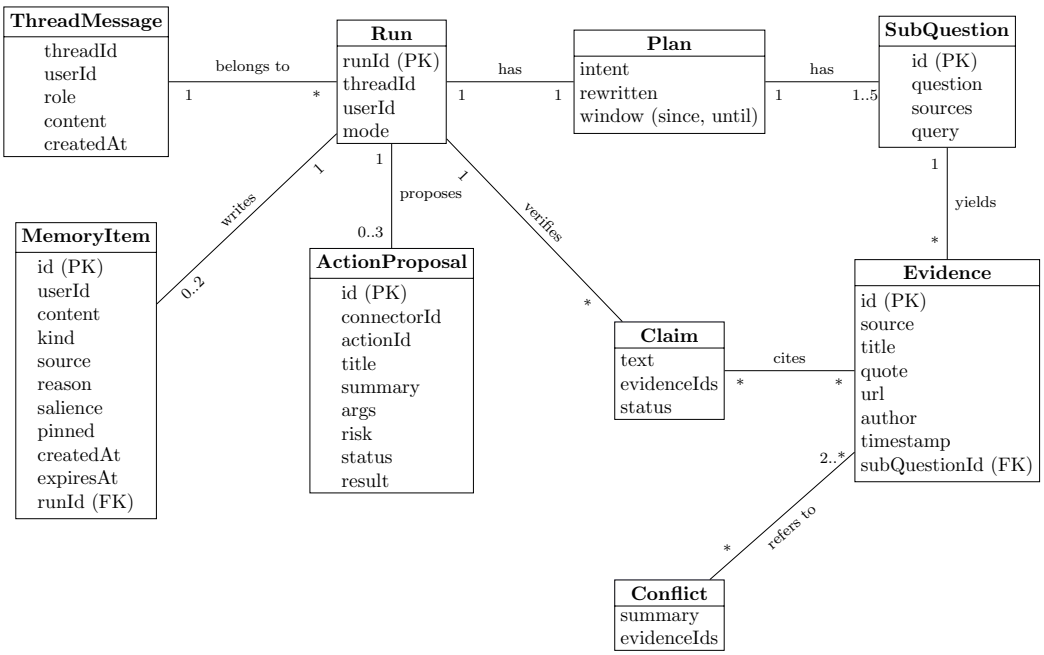


Figure 6.2: Data objects and relationships (ERD form)

6.4.1 Data Flow Diagram

6.4.1.1 Level 0 Data Flow Diagram

Figure 6.3 shows Cortex as a single process with its external entities.

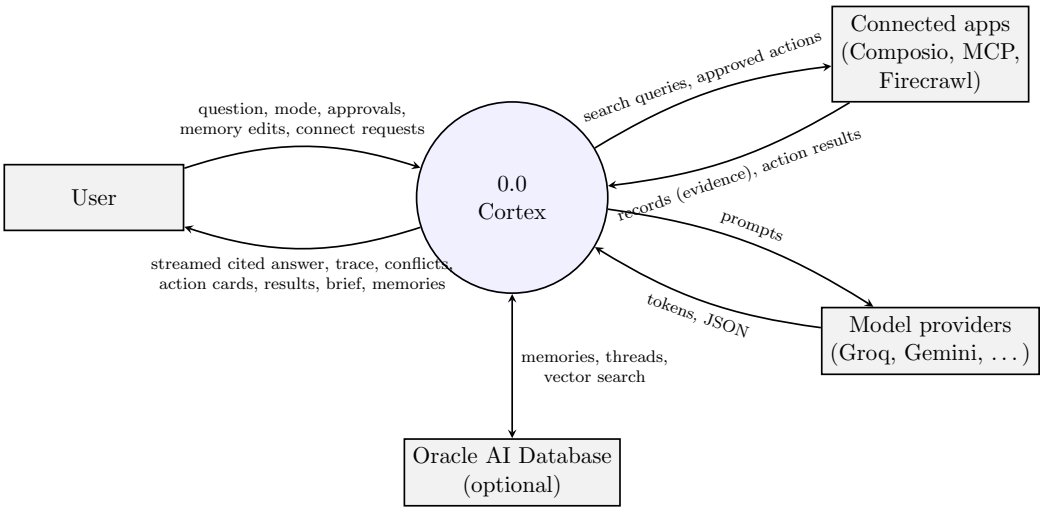


Figure 6.3: Level 0 data flow diagram

6.4.1.2 Level 1 Data Flow Diagram

Figure 6.4 decomposes the process into its nine sub-processes. External entities and data stores marked with an asterisk are repeated to keep lines short. The response cache and the search cache are omitted for clarity.

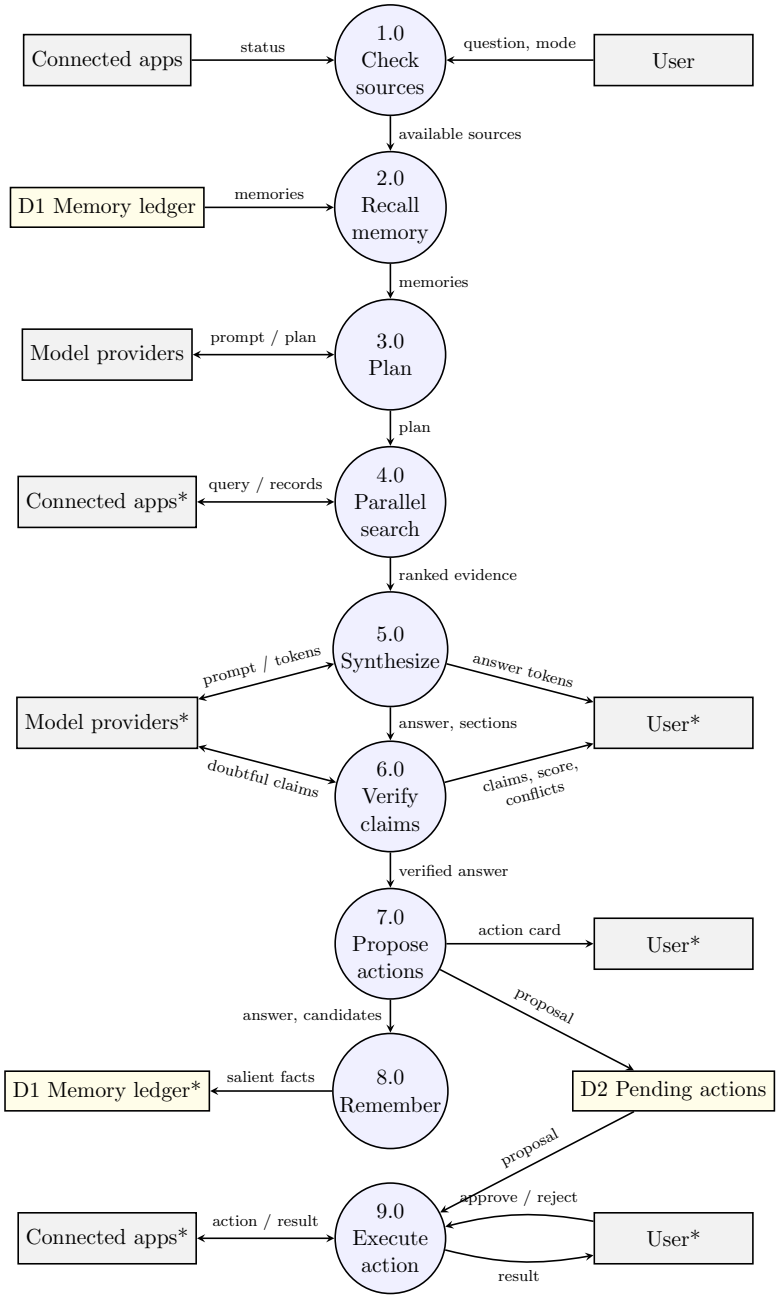


Figure 6.4: Level 1 data flow diagram

6.4.2 Activity Diagram:

- Figure 6.5 follows one run from question to stored memory. The diamonds are the four branch points (Fast mode, small talk, no evidence, action requested), and the bars mark where the searches fork and join.

6.4.3 Non Functional Requirements:

- **Interface Requirements:** a responsive web interface usable on a laptop and on a phone; answers streamed as they are written; the reasoning trace available but collapsed by default; a JSON/HTTP and server-sent event interface between browser and server; an HTTP/JSON interface to the memory service; OpenAI-compatible chat-completion interfaces to model providers; MCP (streamable HTTP or standard input/output) and the Composio SDK to connected apps.

- **Performance Requirements:** the answer should start streaming within a few seconds, and searches run concurrently. Every external call has its own timeout: 12 s for a connector call (with one retry), 6 s for a connector status check, 25 s for a model request and 5 s for a memory request. A whole run is stopped after 90 s. The preliminary evaluation measured a median end-to-end time of 3.7 s and a 90th percentile of 7.1 s in Deep mode.

- **Software quality attributes:**

  - *Availability* (related to reliability): when a model provider fails, the router cools it down and moves to the next one. A failed source is reported and skipped. If the memory service is down the run goes ahead without memory, and if Oracle is down the memory service carries on with SQLite.
  - *Modifiability* (portability, reusability, scalability): adding a source means writing one class that implements the connector interface, or adding one entry to the MCP configuration file. Composio tool names are stored as data and model identifiers as environment variables. The agent, router, connectors and shared types live in separate packages, and the code runs on Windows, Linux and macOS. Horizontal scaling is limited because pending actions and caches are held in the memory of one server process.
  - *Performance*: bounded calls per run; caching of responses (6 h) and searches (90 s).

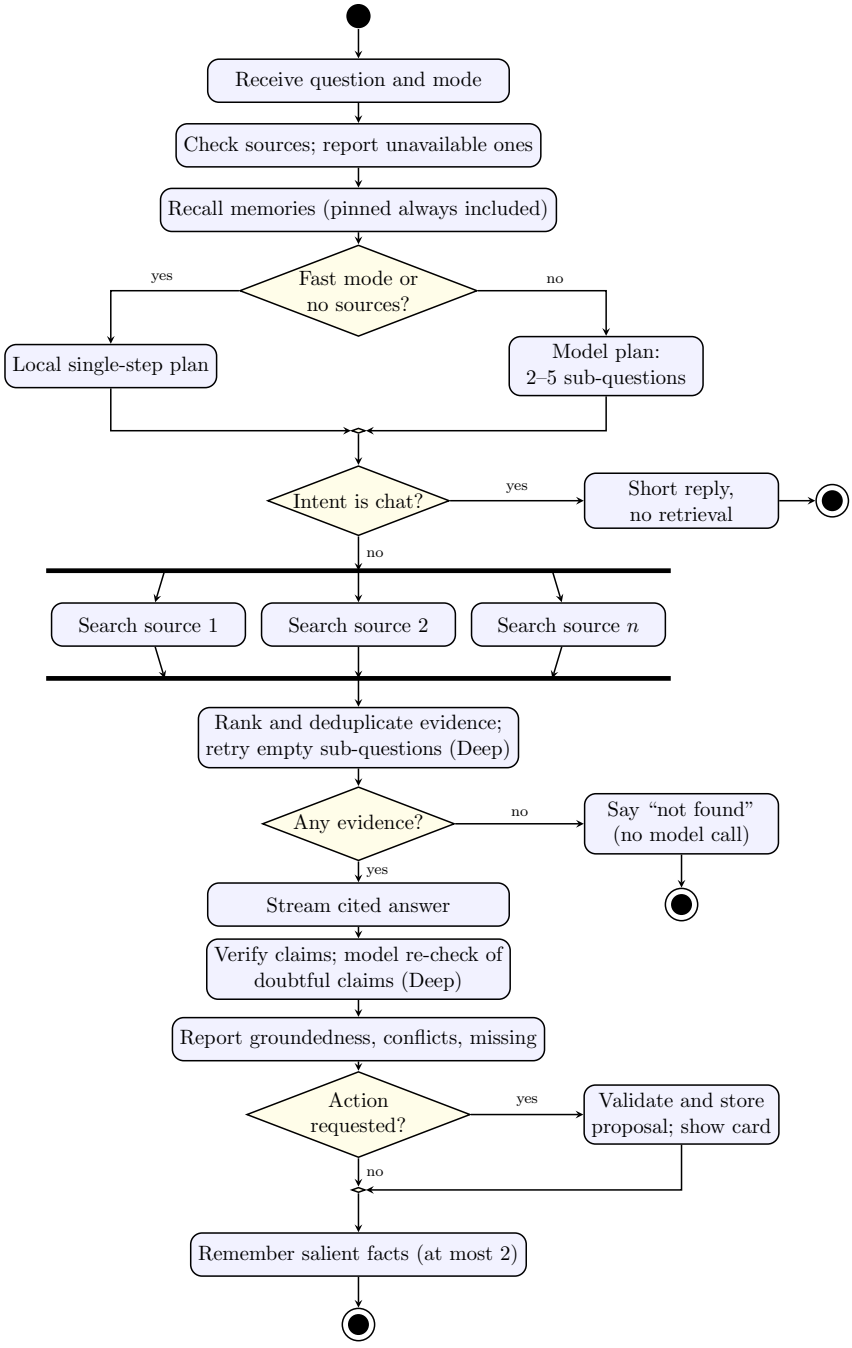


Figure 6.5: Activity diagram of one agent run

– *Security*: an action can run only through the approval endpoint, only once, and only for the user it was proposed to. OAuth tokens stay with Composio and are never sent to the browser. The anonymous identifier travels in an HTTP-only, same-site cookie, and memories from the sample workspace are kept apart from real ones.

– *Testability*: all external dependencies are passed in through one dependency object, so tests replace them with stubs; 18 automated tests; an evaluation harness.

– *Usability*: plain-language failure messages; claims visually marked by verification state; one-click connection; no settings pages or tokens for the end user.

6.4.4 State Diagram:

State Transition Diagram
Fig.6.6 tracks a single action proposal. It is born when the model drafts an action and the store accepts it, and it ends in one of five places: discarded, rejected, expired, executed or failed (double borders). Each arrow is labelled with the event that moves it. There is no arrow from *Pending* straight to *Executed*; the only way through is an explicit approval.

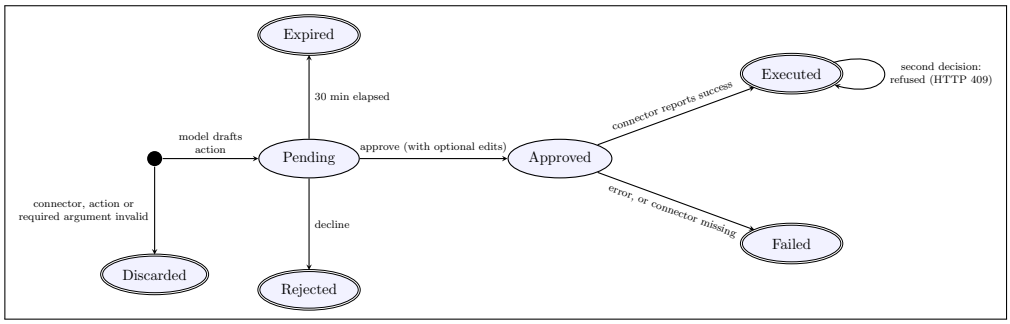


Figure 6.6: State transition diagram

6.4.5 Design Constraints

- Only free-tier services may be used, which bounds model calls per run and requires failover and caching.

- Oracle Agent Memory has only a Python SDK, so memory is a separate Python service.
- Streaming requires a server runtime that keeps the HTTP response open (server-sent events in a Next.js route handler).
- Pending actions are held in the server's memory with a 30-minute lifetime; a restart discards them, which is safe (nothing executes) but requires asking again.
- Evidence quotes are compacted to a few hundred characters to keep prompts small.
- Node.js 24 or later and Python 3.11 are required.

6.4.6 Software Interface Description

Table 6.2 lists every endpoint Cortex exposes or calls. People use the web screens shown in Chapter 10.

Interface	Purpose
Web server (browser to server)	
`POST /api/ask`	Run the agent; response is a server-sent event stream
`POST /api/actions/{id}`	Approve or reject one proposal; the only execution path
`GET /api/brief`	Daily Brief
`GET, POST /api/memory`	List memories; teach a memory
`PATCH, DELETE /api/memory/{id}`	Pin or edit; forget
`GET, PUT /api/connections`	Connection status; choose workspace
`POST /api/connections/{id}`	Start OAuth for one app
Memory service (server to service, HTTP/JSON)	
`GET /health`	Status and active backend (`oracle` or `sqlite`)
`GET, POST /memories`	List; add (near-duplicates reinforce an existing memory)
`PATCH, DELETE /memories/{id}`	Edit or pin; forget
`POST /search`	Recall: pinned memories plus ranked matches
`POST /threads/{id}/messages`	Append conversation turns
`GET /threads/{id}/context`	Thread summary (Oracle) and recent messages
External	
OpenAI-compatible chat completions	Groq, Gemini, Cerebras, OpenRouter, Puter
Composio SDK / hosted MCP	Gmail, Calendar, Drive, Notion, Slack, GitHub tools and OAuth
MCP (HTTP or stdio)	Any server declared in `mcp.config.json`
Firecrawl API	Web search and page scraping
LangSmith API	Optional traces of runs, model calls and searches

Table 6.2: Software interfaces

CHAPTER 7

DETAILED DESIGN DOCUMENT USING APPENDIX A AND B

7.1 Introduction

This chapter describes how Cortex is put together: first the architecture, then the data, then the components. Four rules from our product definition shaped almost every decision. The user should see the evidence along with the answer. Nothing is done in the user's tools without consent. When the evidence is not there, Cortex says "I don't know" rather than guessing. And no loop is allowed to run without a limit.

7.2 Architectural Design

Cortex is a layered system (Figure 7.1). The **browser client** (React) renders the Ask, Brief, Memory and Connections screens and consumes the event stream. The **web server** (Next.js route handlers) authenticates the browser by an anonymous cookie, builds the dependencies for the selected workspace and exposes the API. The **agent package** contains the orchestrator, the prompts, the verifier, the action store, the brief and the tracing wrapper. It depends on three subsystems: the **model router** package, the **connector** package and the **memory service**. A **shared** package holds the types used by both server and client. The orchestrator is written by hand rather than with a graph framework, so that every branch is explicit, every loop is bounded and the number of model calls is predictable.

Package view. The web app, `apps/web`, depends on `packages/agent`, `packages/connectors`, `packages/llm` and `packages/shared`. The agent package in turn depends on the other three. The only link between connectors and `packages/llm` is that the connectors borrow its cache. `services/memory` stands on its own and is reached over HTTP.

Deployment view. During development and evaluation, two processes run on one machine: the Next.js server on port 3000 and the memory service (Uvicorn) on port 8000. The optional Oracle AI Database Free runs in a Docker container on port 1521. All other services are remote APIs. Deployment to a hosted environment is planned for Term II.

7.3 Data design (using Appendices A and B)

The data structures are deliberately small and typed. All types shared by server and client are defined once in the shared package, and the structured outputs of the models are validated with schemas before use.

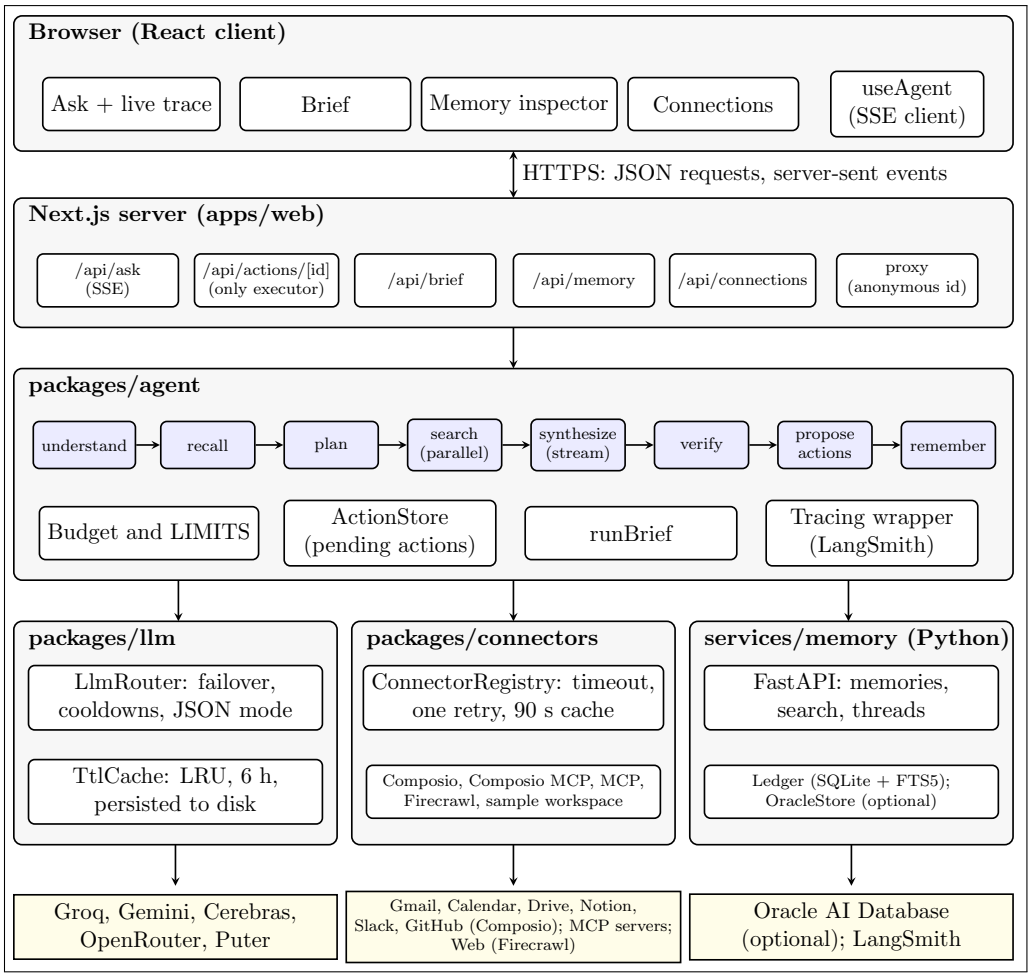


Figure 7.1: Architecture diagram

7.3.1 Internal software data structure

These structures travel between the packages:

- **AgentEvent** stream: an asynchronous generator of typed events produced by the orchestrator, serialised by the ask route as `data: {json}` lines and folded by the client hook into a *Turn* (steps, plan, tool calls, evidence, answer, claims, conflicts, actions, memories, usage).
- **SearchInput** (`query`, `since`, `until`, `limit`) and **RawEvidence** (evidence without run-scoped identifiers), exchanged between the agent and the connectors.
- **SafeSearchResult**: either {ok, items, cached, ms} or {failure, ms}; connector errors never escape as exceptions.
- **CompleteOptions** and **Completion**: model tier (`strong` or `fast`), messages, JSON flag, token limit, cache lifetime and abort signal; and the returned text, provider, model and cache flag.
- **Sections**: the synthesis output split at fixed markers into the answer, conflicts, missing items, action drafts (JSON), memory candidates (JSON) and follow-up questions.

7.3.2 Global data structure

These are shared across most of the server:

- Process-wide singletons on the server: one model router (with its provider health map and response cache), one action store, one memory client and one connector registry per workspace (sample or live). They survive development hot reloads.
- The `LIMITS` constant of the agent (sub-questions, tool calls, model calls and evidence per mode, run timeout).
- The provider list, built once from environment variables in failover order (free tiers first, Puter last).
- The toolkit specifications: for each app, its search tool, the mapping of search input to tool arguments, and its actions with parameters, risk and preview function.

7.3.3 Temporary data structure

Cortex writes these short-lived files:

- `.cache/llm-cache.json`: the persisted response cache of the model router (entries expire; least recently used entries are evicted beyond 1000).
- `.cache/eval-llm-cache.json`: the response cache used by evaluation runs.
- `evals/results/<mode>-<date>.json`: the per-question rows and the summary of an evaluation run.
- In memory only: the search cache (90 s), the pending actions (30 min) and the per-run budget.

7.3.4 Database description

The memory service uses an SQLite database file (`services/memory/data/ledger.db`) with the tables in Table 7.1, and an FTS5 full-text index `memories_fts(id, content)` used for keyword search with BM25 ranking when Oracle is not configured. When Oracle AI Database is configured, the same memories are also written to an Oracle Agent Memory store (identifier `CORTEX`) for vector search, with automatic extraction switched off, and conversation threads are kept there for summaries. The ledger remains the source of the governance fields (reason, pin, salience, expiry) in both cases.

7.4 Component Design

The main components are described below; their classes and relationships are shown in Figure 7.2, and the interaction for a consent-gated action in Figure 7.3. The algorithms are given in Chapter 8.

- **Orchestrator** (`runAgent`): runs the eight steps of a run as an asynchronous generator, enforces the budget, converts every failure into an event, and stamps each step with its duration.
- **Prompts**: the planning prompt (returns a JSON plan), the synthesis prompt (evidence-only, cite every factual sentence, followed by five marked sections), the verification prompt, the chat prompt and the brief prompt.

Table	Column	Description
memories	id (PK)	UUID
	user_id	Owner (workspace-scoped); indexed
	content	The remembered text (3–500 characters)
	kind	priority, preference, fact, person or decision
	source	Where it came from (e.g. conversation)
	reason	Why it was kept; "edited by you" appended on edit
	salience	Stored salience in $[0, 1]$
	pinned	1 if pinned (no decay, no expiry)
	created_at	ISO timestamp; refreshed on reinforcement
	expires_at	ISO timestamp or NULL (never)
	run_id	Run that created the memory
messages	thread_id	Conversation; indexed
	user_id	Owner
	role	user or assistant
	content	Message text (first 4000 characters)
	created_at	ISO timestamp

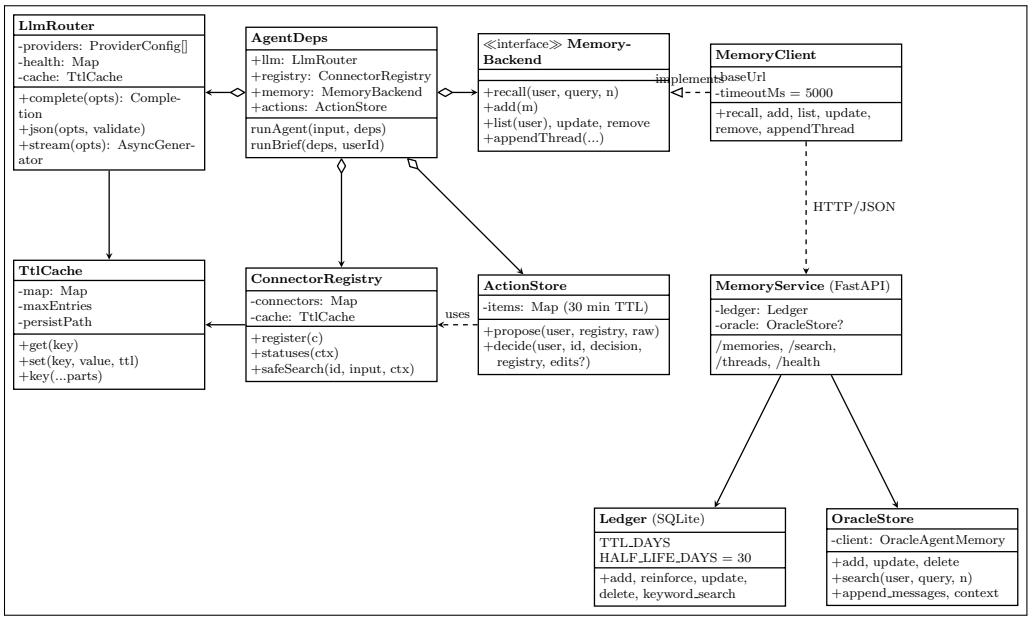
Table 7.1: Database tables of the memory service

- **Verifier**: splits the answer into claim units (shared with the client so that both agree on what a claim is), checks citations, overlap and numbers, computes groundedness, and maps conflict lines to evidence.
- **ActionStore**: validates and stores proposals and executes an approved proposal once.
- **LlmRouter** and **TtlCache**: provider failover, cooldowns, JSON parsing, streaming with failover before the first token, and caching.
- **ConnectorRegistry** and the connectors: a uniform search and execute interface over Composio (SDK or hosted MCP), any MCP server, Firecrawl and the sample workspace, with a tolerant evidence extractor.
- **Memory client and service**: HTTP client in the agent; FastAPI service with the SQLite ledger and the optional Oracle store.

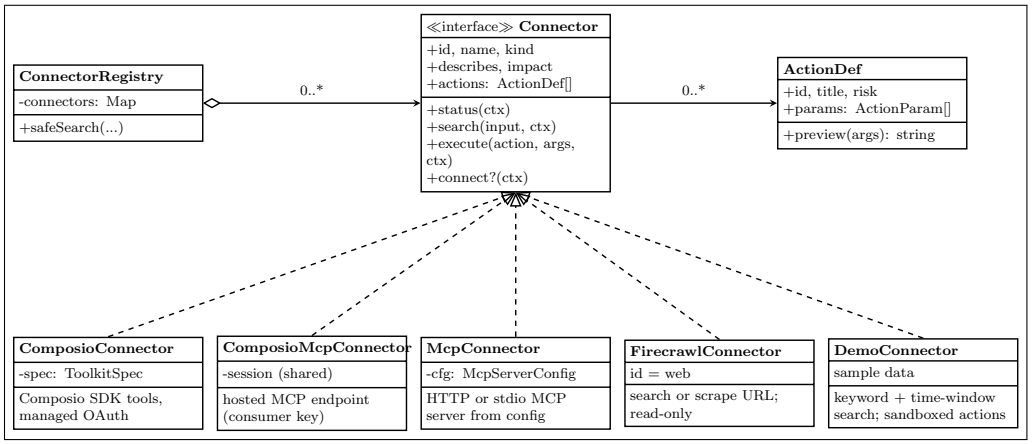
7.4.1 Class Diagram

7.4.2 Interaction diagram for a consent-gated action

Figure 7.3 shows the sequence of messages from a question that asks for an action to its execution. The agent's run ends after the proposal is stored; execution happens in a separate request, triggered only by the user.



(a) Agent, model router, actions and memory



(b) Connector layer

Figure 7.2: Class Diagram

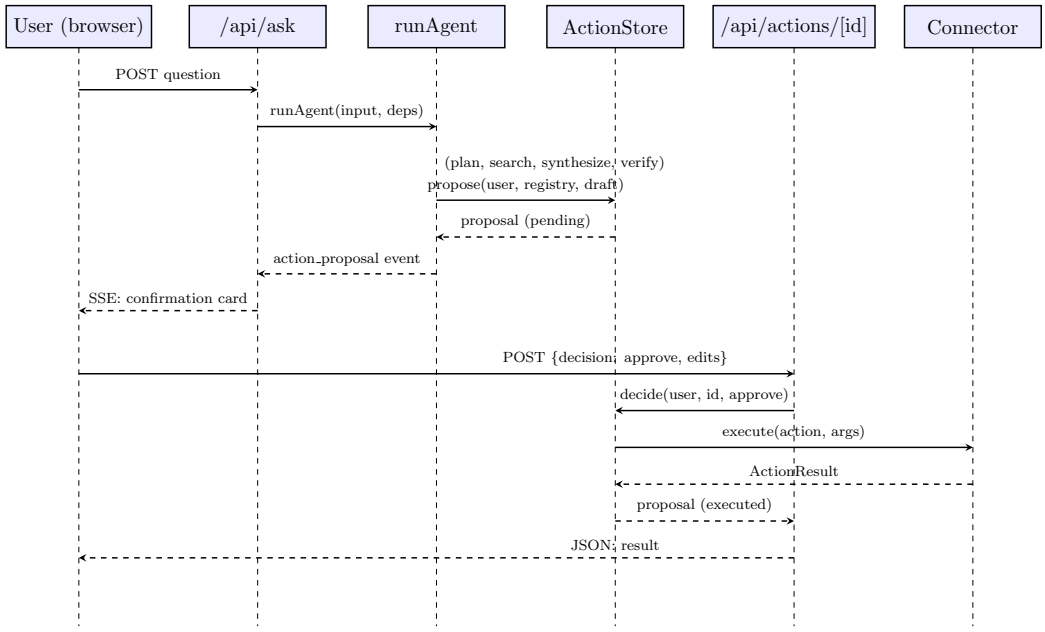


Figure 7.3: Sequence diagram of a consent-gated action

CHAPTER 8
PROJECT IMPLEMENTATION

8.1 Introduction

Cortex is implemented as a TypeScript monorepository with npm workspaces (`apps/web`, `packages/agent`, `packages/connectors`, `packages/llm`, `packages/shared`) and a Python memory service (`services/memory`), together with an evaluation harness (`evals`) and helper scripts (`scripts`). The delivered code is about 6.1 thousand non-blank lines. Below we list the tools and technologies we used and give the main algorithms in pseudo code. Each algorithm was written from the actual code in the repository, so the two can be read side by side.

8.2 Tools and Technologies Used

8.3 Methodologies/Algorithm Details

The orchestration follows a plan-then-execute methodology with fixed budgets. Algorithm 1 is the agent run, Algorithm 2 the claim verification, Algorithm 3 the model router's failover, and Algorithm 4 the memory policy.

Layer	Technology	Use in Cortex
Web application	Next.js 16, React 19, TypeScript	Screens, API route handlers, server-sent event streaming
Styling and motion	Tailwind CSS 4, Motion, Lucide icons	Interface styling, trace reveal animations, icons
Validation	Zod	Schemas for the plan, memory candidates, verification and brief outputs
Model access	OpenAI-compatible client	One client for Groq, Gemini, Cerebras, OpenRouter and Puter
Connectors	Composio SDK, MCP TypeScript SDK, Firecrawl SDK	Six work apps with managed OAuth; any MCP server; web search and scraping
Memory service	Python 3.11, FastAPI, Uvicorn, Pydantic, SQLite (FTS5)	Memory API, ledger and keyword search
Optional memory backend	Oracle AI Database, `oracledb`, Oracle Agent Memory	Vector search over memories, conversation threads
Observability	LangSmith SDK	Optional traces: run, model calls, searches
Testing	Vitest; `tsx` evaluation runner	18 automated tests; 12-question evaluation
Interface review	Playwright (screenshots)	Desktop and mobile screenshots used in Chapter 10

Table 8.1: Tools and technologies

8.3.1 Algorithm 1/Pseudo Code

Algorithm 1: Bounded agent run (runAgent)

Input: question q, mode m, user u, history H
Output: ordered stream of typed events
$C \leftarrow$ connected sources;
emit a failure event (source, reason, impact) for each unavailable source;
$M \leftarrow \text{recall}(u, q, 5)$; `// pinned memories always included`
if $m = \textit{fast}$ ***or*** $C = \emptyset$ **then**
    $\Pi \leftarrow$ one sub-question over all sources (web only if hinted), keywords of q, locally parsed time window;
else
    $\Pi \leftarrow$ JSON plan from the fast model tier, validated and limited to 5 sub-questions over known sources;
    if *the output is unusable* **then**
        $\Pi \leftarrow$ the local single-step plan;
if $\textit{intent}(\Pi) = \textit{chat}$ **then**
    stream a short reply without retrieval;
    return;
$T \leftarrow$ deduplicated pairs (source, query) of all sub-questions, $|T| \leq 18$;
run safeSearch for every $t \in T$ concurrently; collect evidence E; emit one failure event per failed source;
if $m = \textit{deep}$ **then**
    foreach *sub-question without evidence* **do**
        search its working sources again with the first two keywords and no time window, within the remaining tool budget;
$E^* \leftarrow$ the top k items of E by score after deduplication ($k = 16$ Deep, 8 Fast), numbered $e_1, e_2, \ldots$;
if $E^* = \emptyset$ **then**
    emit an honest "not found" answer naming the sources searched, without a model call;
    return;
stream the synthesis (strong tier in Deep mode, fast tier in Fast mode), forwarding tokens only up to the first section marker;
split the output into answer, conflicts, missing, actions, memory and follow-ups;
$(L, G) \leftarrow \text{VerifyClaims}(\textit{answer}, E^*)$;
if $m = \textit{deep}$ ***and*** *some cited claims are not supported* ***and*** *the model budget allows* **then**
    re-judge up to 8 doubtful claims with the fast tier; keep the lexical verdicts if this fails;
emit the verdict: claims, G, conflicts mapped to evidence, missing items, follow-ups;
foreach *action draft (at most 3)* **do**
    if *ActionStore.propose succeeds* **then**
        emit the proposal with status pending;

8.3.2 Algorithm 2/Pseudo Code

Algorithm 2: Claim verification and groundedness (VerifyClaims)

Input: answer A, ranked evidence E^*
Output: claims L, groundedness G
$L \leftarrow \emptyset$; $P \leftarrow \emptyset$; `// evidence cited by the previous sentence`
foreach *claim unit c of A (sentences and list items; code, quotes and remarks about Cortex's own drafts excluded)* **do**
  $R \leftarrow$ the citation numbers $[n]$ in c;
  if $R = \emptyset$ **then**
    if *c states no fact **or** c is a hedge (e.g. "the evidence does not mention")* **then**
      skip c;
    if $P \neq \emptyset$ ***and*** $\mathit{Score}(c, P) = \mathit{supported}$ **then**
      append $(c, P, \mathit{supported})$ to L;
    else
      append $(c, \emptyset, \mathit{unsupported})$ to L;
  else
    $E_c \leftarrow \{E^*_n : n \in R\}$;
    if $E_c = \emptyset$ **then**
      append $(c, \emptyset, \mathit{unsupported})$ to L;
    else
      $P \leftarrow E_c$; append $(c, E_c, \mathrm{Score}(c, E_c))$ to L;
$G \leftarrow 1$ if $L = \emptyset$, otherwise $\frac{1}{|L|} \sum_{c \in L} w(c)$ with $w = 1, 0.5, 0$;
return (L, G);

Score(c, E_c): $T(\cdot)$ lower-cases, removes stop words and short tokens and strips common suffixes;
$o \leftarrow |T(c) \cap T(E_c)| \,/\, |T(c)|$; *numbersOk* $\leftarrow$ every number in c occurs in E_c;
return *supported if $o \geq 0.4$ and numbersOk; partial if $o \geq 0.2$; unsupported otherwise*;

8.3.3 Algorithm 3/Pseudo Code

Algorithm 3: Free-tier model router with failover (LlmRouter.complete)

```
Input: tier (strong or fast), messages, options
Output: completion, or LlmUnavailableError
key ← SHA-256 of (tier, messages, JSON flag, token limit);
if the cache holds a live entry for key then
    return the cached completion (marked cached);
for round ← 1 to 2 do
    if round = 2 then
        wait for the provider that recovers soonest if that is within
         12 s; otherwise stop;
    foreach provider p not cooling down, in order (free tiers first,
     Puter last) do
        request the model of the tier from p (25 s timeout);
        if the reply is non-empty and, if JSON was requested,
         parseable then
            reset the health of p; cache the reply (default 6 h);
            return the reply;
        else
            cooldown of p: rate limit (429) → retry-after or 60 s;
             invalid key (401/403) → 10 min; out of credit (402) →
             1 h; model not found (404) → 10 min; otherwise
             15 s × min(failures, 4);
raise LlmUnavailableError listing every provider and reason;
```

Streaming uses the same loop, but fails over only before the first token: a stream that breaks after it has started is reported honestly rather than silently restarted on another provider.

8.3.4 Algorithm 4/Pseudo Code

Algorithm 4: Memory policy: write and recall

Input: memory candidate m (user, content, kind, salience, reason, source, run); or a recall request (u, q, n)

Write(m):;

if *a live memory of the same user has similarity ratio* > 0.82 *with* m **then**

  reinforce it: salience $\leftarrow \max(old, new)$, creation time $\leftarrow$ now;

else

  insert m with expiry from its kind (priority 45 d, decision 120 d, fact 90 d, person and preference never);

  if Oracle is configured, add it to Oracle Agent Memory with the same identifier;

Recall(u, q, n):;

$R \leftarrow$ up to 3 pinned memories of u;

$H \leftarrow$ Oracle vector search for $2n$ hits; if unavailable or empty, FTS5 BM25 search (or token overlap) for $2n$ hits;

foreach *hit* $h \in H$ *not already in* R*, live and not expired* **do**

  $s_{eff} \leftarrow s$ if pinned, else $s \cdot 0.5^{\,age/30}$;

  $score(h) \leftarrow rel(h) \cdot (0.5 + 0.5\, s_{eff})$;

return R *followed by the best* $n - |R|$ *hits by score*;

8.4 Verification and Validation for Acceptance

The system is accepted against the goals and objectives of Section 1.8 with the following checks; Table 8.2 gives the status at the time of writing.

Acceptance criterion	How it is checked	Status
Every factual sentence cites evidence; claims are verified	Verification unit test; groundedness in evaluation	Met (mean 0.88)
Expected facts are found in the cited evidence	Citation recall in evaluation	Met (0.92)
Unanswerable questions are answered honestly	"Not found" unit test and evaluation case	Met (1/1)
No action executes without explicit approval	ActionStore and Fast Mode unit tests; evaluation check	Met (0 executions without consent)
Cross-source conflicts are reported	Unit test; two evaluation cases	Partly met (1/2)
Requested actions are proposed	Two evaluation cases	Partly met (1/2)
A failed source is named and the run continues	Integration test with a failing connector	Met
Model provider failure is handled	Router failover and cooldown tests	Met
Fast Mode uses one model call	Unit test	Met
Real services work with real keys	Live checks of Groq, Gemini (Flash-Lite), Firecrawl, LangSmith and Composio hosted MCP	Met
Oracle memory backend	Local Oracle AI Database Free container	Not met yet (SQLite fallback in use)

Table 8.2: Acceptance checks

CHAPTER 9
SOFTWARE TESTING

9.1 Type of Testing Used

- **Unit testing** (Vitest): pure functions and single classes, such as JSON parsing, claim verification, section splitting, time-window parsing, evidence extraction and the action store.
- **Integration testing** (Vitest): the model router against local stub HTTP servers that imitate providers (including a rate-limited one); the connector registry with a failing connector; and the whole agent run over the sample workspace with a stubbed model, checking the event stream end to end.
- **System testing and evaluation**: the evaluation harness runs 12 questions end to end in Deep mode over the sample workspace with real free-tier models and measures groundedness, citation recall, conflict detection, honest "not found" answers, action proposals, execution without consent, and latency.
- **Live service checks**: a script lists the models of every configured provider and sends a JSON request to each tier, verifies every Composio tool used by the toolkits, and checks Firecrawl, printing names and status only, never keys.
- **Interface review**: desktop and mobile screenshots of every screen were reviewed against the design brief, and the issues found were fixed in two review rounds.

9.2 Test Cases and Test Results

All 18 automated tests pass. Tables 9.1, 9.2 and 9.3 list them by package. The evaluation results follow in Table 9.4.

Evaluation. The evaluation used Deep mode over the fictional sample workspace with free-tier models: Groq `openai/gpt-oss-120b` as the primary strong model (with `gpt-oss-20b` as the fast tier) and Gemini 3.5 Flash-Lite as the fallback. Questions were spaced six seconds apart. Because the router tries Groq first, the eight questions answered by Gemini are those for which Groq was rate-limited or failing at that moment. Table 9.4 gives the per-question results. These results are *preliminary*: twelve questions over one small fictional workspace are not a benchmark, and repeated runs differ (conflict detection in particular varied between runs).

ID	Test case	Expected result	Result
TL1	Two providers, the first returns HTTP 429; then a second request	Answer comes from the healthy provider; the limited provider is not called again while cooling down	Pass
TL2	The same prompt is sent twice	Second answer is served from the cache; the provider is called once	Pass
TL3	No provider is configured	`LlmUnavailableError` is raised	Pass
TL4	JSON inside a code fence, or embedded in prose	Both are parsed into the same object	Pass

Table 9.1: Test cases: model router (packages/llm)

ID	Test case	Expected result	Result
TC1	Search Slack and GitHub of the sample workspace for Demo Day	Slack says Demo Day moved; GitHub returns the milestone (the planted conflict)	Pass
TC2	Empty query with a one-week time window	Recent items are returned instead of nothing	Pass
TC3	Execute "send email" on the sample Gmail connector	Recorded in the sandbox; message says nothing left the sample workspace	Pass
TC4	A connector whose search always fails with "fetch failed"	Failure names Slack, reason "could not be reached", impact about blockers; exactly two attempts	Pass
TC5	Nested tool output (data, messages, subject, text, timestamp, sender)	Evidence with title, quote, author and normalised timestamp	Pass

Table 9.2: Test cases: connectors (packages/connectors)

ID	Test case	Expected result	Result
TA1	Fast Mode run over the sample workspace with a stub model	Exactly one model call; plan and tool calls emitted; no section markers reach the user; one conflict; proposal pending; stream ends with done	Pass
TA2	Run with Slack failing	A source error names Slack; an answer is still produced	Pass
TA3	Question with no matching evidence	"Couldn't find" answer; zero model calls	Pass
TA4	"Remember that Orbit is my top priority . . . "	The memory is written	Pass
TA5	Claims with a correct number, a wrong number, and no citation	supported, partial, unsupported	Pass
TA6	Synthesis output with sections; stream ending in a partial marker	One action parsed; partial marker held back	Pass
TA7	"this week", "upcoming deadlines", "hello"	last 7 days, next 30 days, no window	Pass
TA8	Approve by another user; approve by the owner; approve again	Refused; executed; refused as already executed	Pass
TA9	Proposal missing a required field	Rejected naming the missing field	Pass

Table 9.3: Test cases: agent, verification and actions (packages/agent)

Question ID	Groundedness	Citation recall	Specific check	Time (s)	Answered by
demo-day-change	0.89	1.00	conflict: yes	5.4	Groq
blockers	1.00	1.00	–	7.1	Gemini
login-bug	0.81	1.00	–	3.7	Groq
report-due	0.67	0.50	–	4.7	Gemini
oa-deadline	0.50	0.50	–	3.3	Gemini
user-testing	1.00	1.00	–	3.5	Groq
chat-scope	1.00	1.00	–	8.0	Gemini
oss-pr	0.67	1.00	–	2.8	Gemini
risks-week	1.00	1.00	conflict: no	4.9	Gemini
unknown	1.00	1.00	not found: yes	2.4	Gemini
action-slack	1.00	1.00	action: yes	2.4	Gemini
action-email	1.00	1.00	action: no	3.2	Gemini
Summary	**0.88**	**0.92**	see text	p50 3.7, p90 7.1	

Table 9.4: Preliminary evaluation results (Deep mode, 12 questions)

In summary, mean groundedness was 0.88 and citation recall 0.92. The one unanswerable question got an honest "not found" (1/1), one of two expected actions was proposed (1/2), and one of two planted conflicts was detected (1/2). No action was executed without consent. Median latency was 3.7 s, with a 90th percentile of 7.1 s. Citation recall is the fraction of expected key facts that appear in the evidence the answer actually cites.

CHAPTER 10

RESULTS

10.1 Screen shots

The screenshots below show the main screens over the sample workspace. They were captured for the interface review with the development mock model, which quotes evidence verbatim, so they show the interface and its states rather than the quality of a real model's answer; the measured quality is reported in Section 10.2.

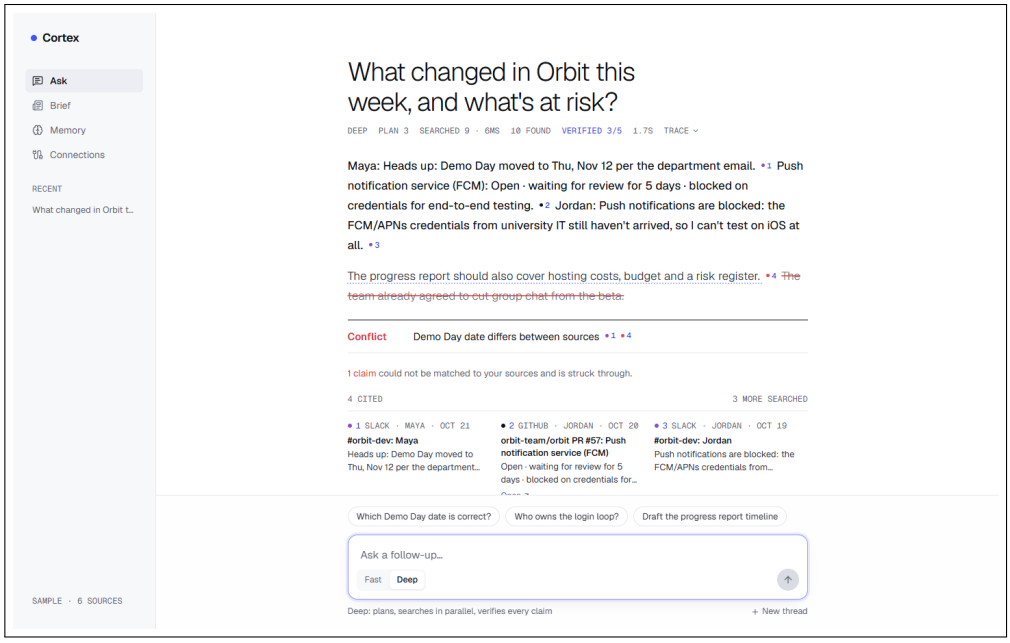


Figure 10.1: Ask screen: a cited answer with a struck-through unsupported claim, a reported conflict between two sources, and the cited sources

10.2 Outputs

For one question the system produces the streamed answer with numbered citations, followed by a verdict that gives the status of each claim, the groundedness, conflicts, missing items and follow-up questions. If the run proposed actions or wrote memories, those come next. A usage record closes the run with the model calls, cached calls, tool calls, time and models used. Figures 10.8 and 10.9 summarise the measured outputs of the preliminary evaluation in Table 9.4.

Discussion. Groundedness was 1.00 for seven of the twelve questions. The lowest values (0.50 and 0.67) came from questions about deadlines ("oa-

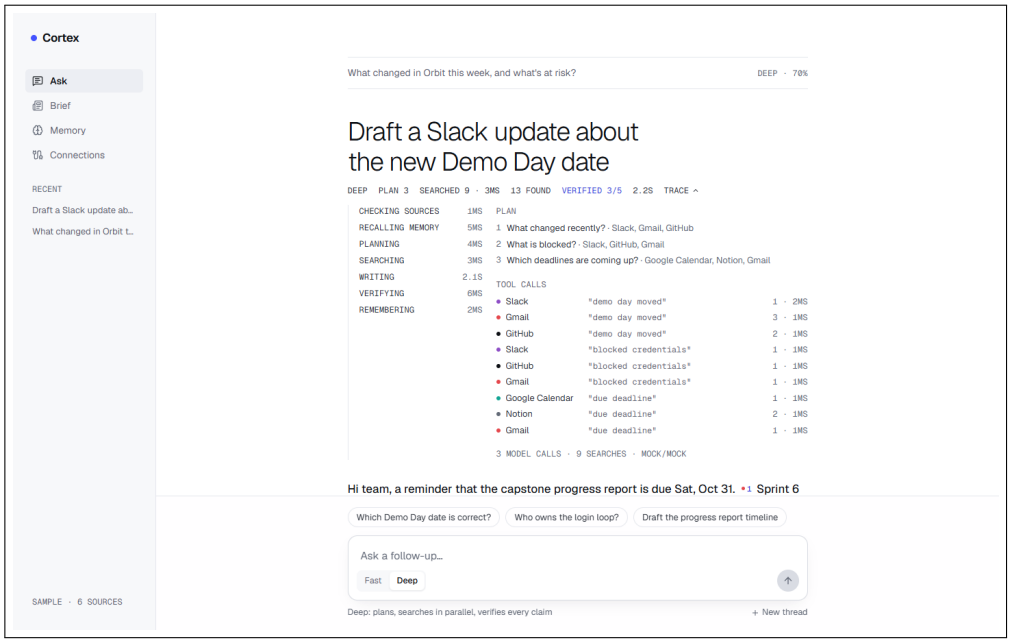


Figure 10.2: Live trace: step timings, the three-step plan and each tool call with its query, result count and latency

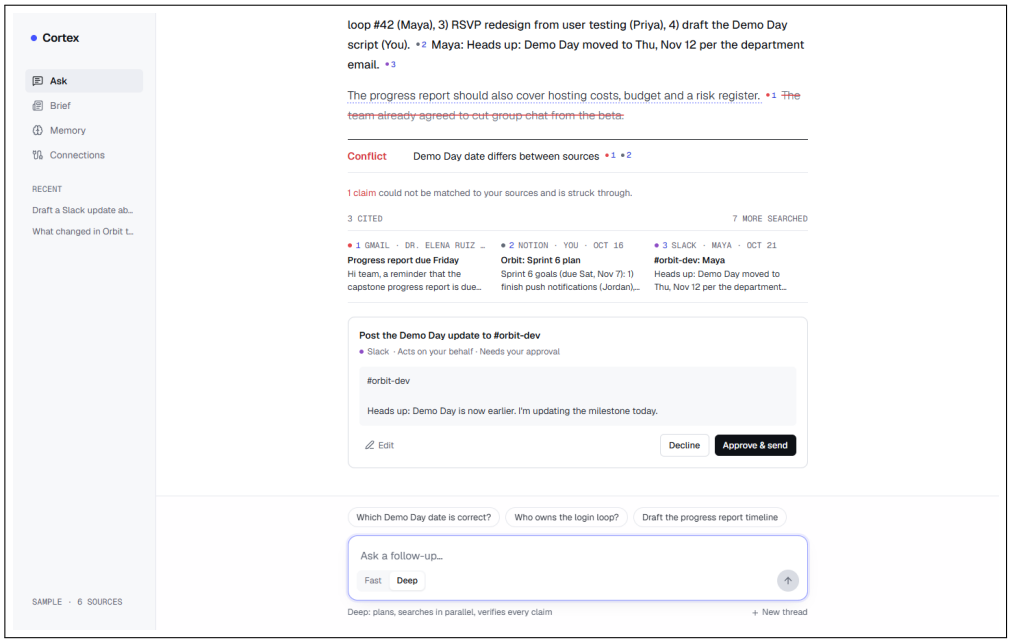


Figure 10.3: A proposed Slack message waiting for approval, with an exact preview and Edit, Decline and Approve controls

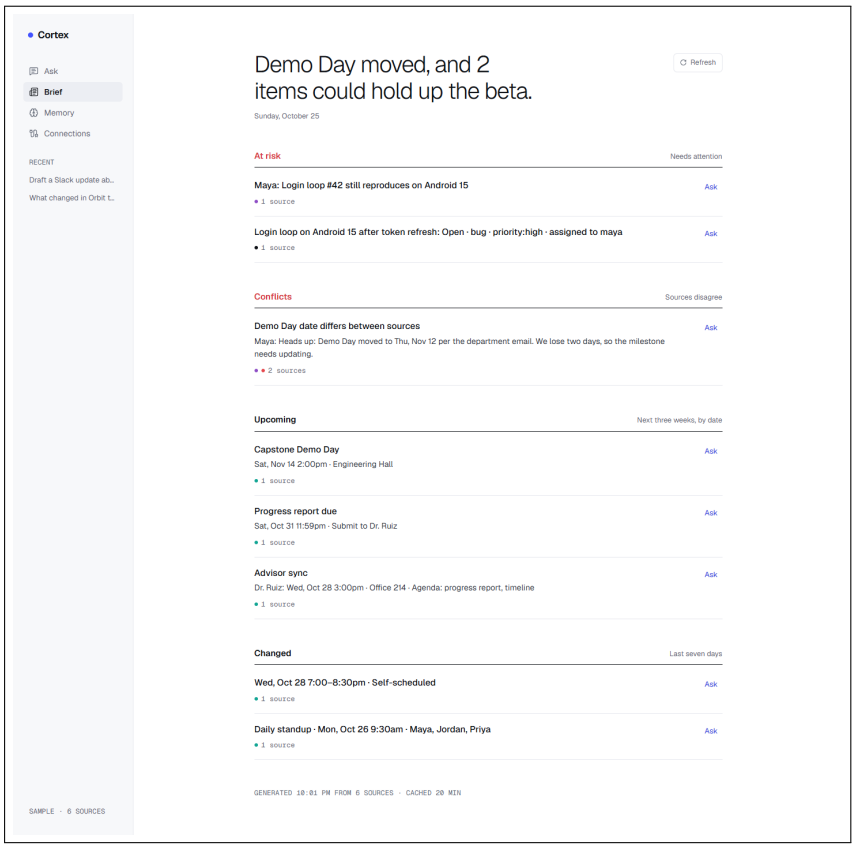


Figure 10.4: Daily Brief: at-risk items, conflicts, upcoming deadlines and recent changes, each linked to its sources

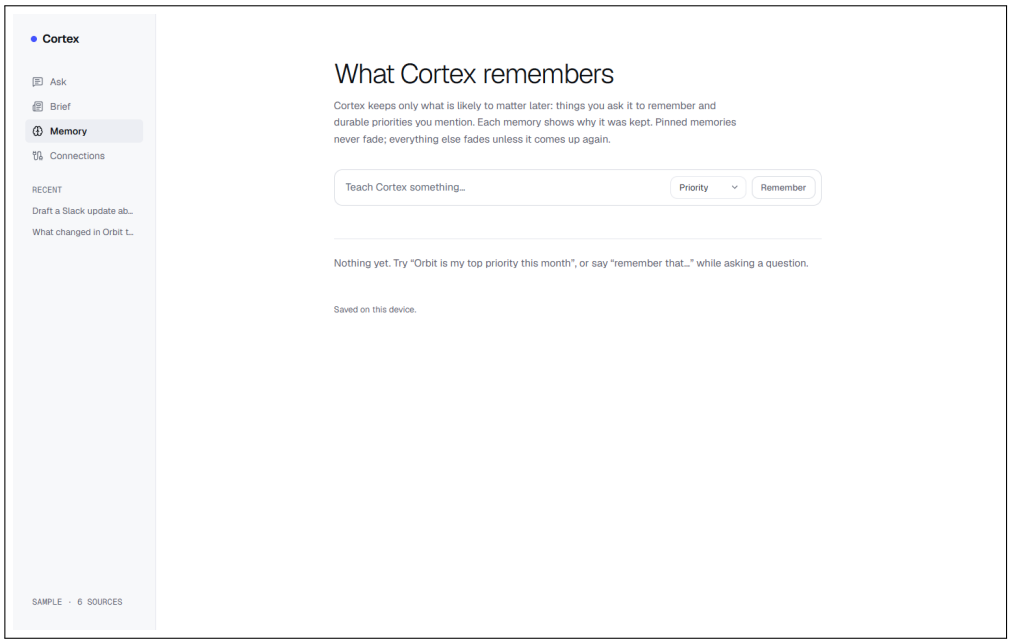


Figure 10.5: Memory inspector (empty state) with the control to teach Cortex a memory of a chosen kind

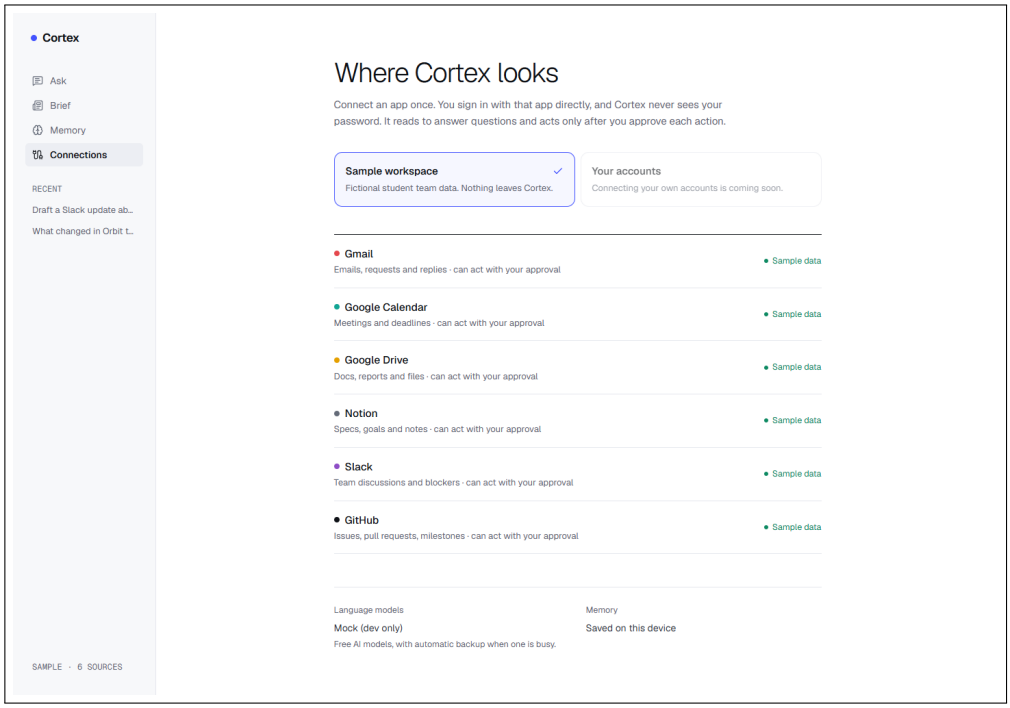
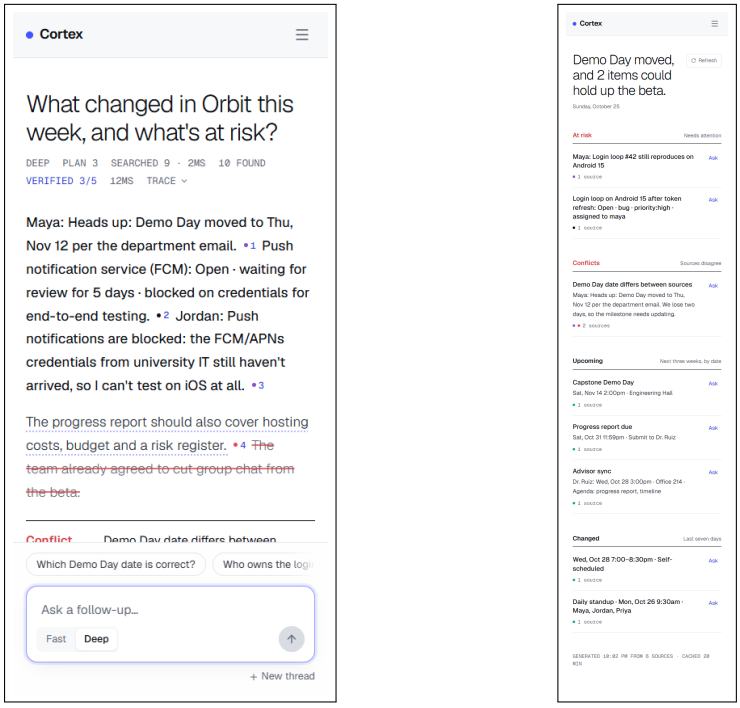


Figure 10.6: Connections: the sample workspace with six sources, and the language model and memory status



(a) Ask (b) Brief

Figure 10.7: Mobile layout

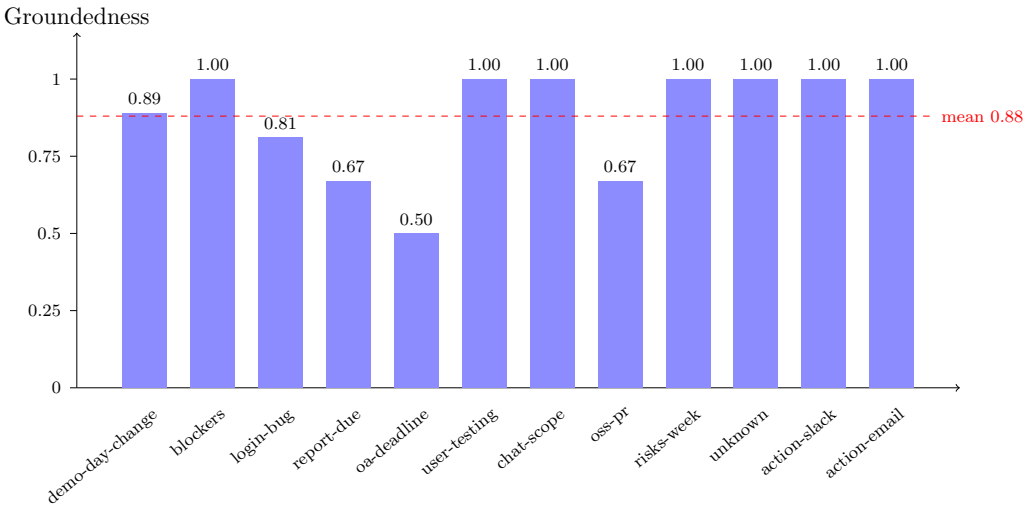


Figure 10.8: Groundedness per evaluation question

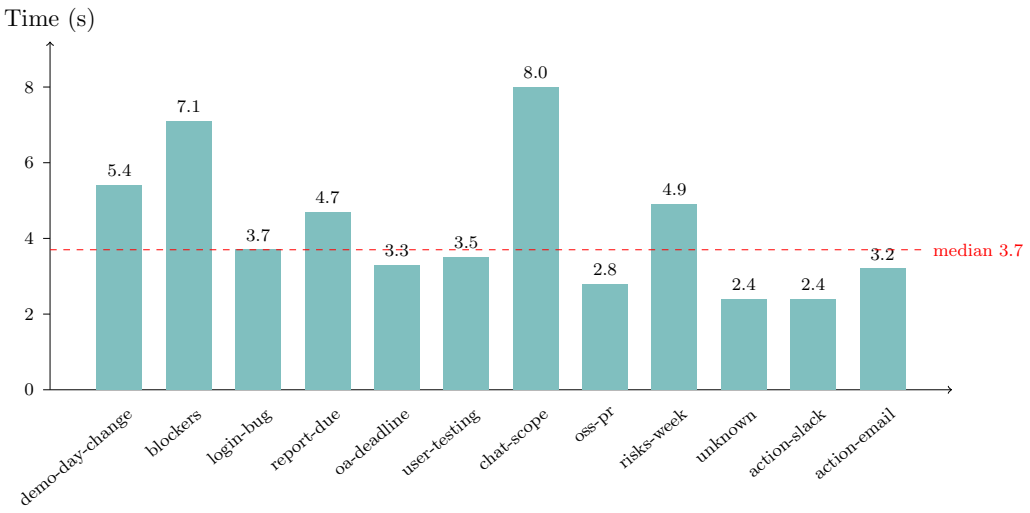


Figure 10.9: End-to-end time per evaluation question (Deep mode)

deadline", "report-due"), where the answers added dates or details in sentences that the lexical check could not match to the cited evidence. The same two questions had a citation recall of 0.5. We planted a Demo Day conflict and wrote two questions to expose it. The direct question surfaced it, but the broader weekly summary did not, and the behaviour changed from run to run. At present conflict detection depends on the model filling in the conflicts section, and this is the weakness we most need to fix. Of the two action requests, only the Slack one led to a proposal; the e-mail request produced no Gmail proposal in this run. In all twelve runs no action was executed, which is the property the design guarantees regardless of model behaviour. The unanswerable question about a company's Series B was answered as not found. Live tests with real keys confirmed that Groq, Gemini (Flash-Lite), Firecrawl, LangSmith and Composio's hosted MCP endpoint (consumer key) work with the system; the Oracle AI Database container was set up but could not be reached because of a Docker networking issue, so all memory results use the SQLite backend.

CHAPTER 11

DEPLOYMENT AND MAINTENANCE

11.1 Installation and un-installation

Prerequisites: Node.js 24 or later, npm, Python 3.11 and Git; optionally Docker for the local Oracle database.

Installation:

1. Obtain the source code and run `npm install` in the repository root.
2. Copy `.env.example` to `.env` and add at least one model key (for example `GEMINI_API_KEY` or `GROQ_API_KEY`). Optional keys: `COMPOSIO_API_KEY` (own accounts), `FIRECRAWL_API_KEY` (web), `LANGSMITH_API_KEY` (tracing), and the Oracle connection settings.
3. Run `npm run memory:setup` once to create the Python virtual environment of the memory service.
4. Start the memory service with `npm run memory` (port 8000) and, in a second terminal, the web application with `npm run dev` (`http://localhost:3000`).
5. Optional: `npm run oracle:local` starts a local Oracle AI Database Free container; set `ORACLE_DB_USER`, `ORACLE_DB_PASSWORD` and `ORACLE_DB_DSN` so that the memory service uses it.
6. Optional: to add an MCP server, copy `mcp.config.example.json` to `mcp.config.json`, edit it and set `CORTEX_MCP_CONFIG`.
7. Verification: `npm test` runs the 18 tests; `npm run eval` runs the evaluation; `npx tsx scripts/check-services.ts` checks the configured services.

Without any keys, `npm run mock-llm` starts a development-only mock model; the application then uses it when `CORTEX_MOCK_LLM_URL` is set.

Un-installation: stop both processes; delete the repository folder (this removes the dependencies, the virtual environment, the caches in `.cache` and the memory ledger in `services/memory/data`); if the Oracle container was created, remove it with `docker rm -f cortex-oracle` and its volume with `docker volume rm cortex-oradata`; revoke any API keys and Composio connections that are no longer needed.

Maintenance: free-tier model catalogues change without notice. For that reason model identifiers live in environment variables, and the service check script reports any model that has disappeared. Composio tool names are kept together in one file. Connector failures show up in the interface and,

if enabled, in LangSmith traces. Whenever prompts or models change, we re-run the evaluation.

11.2 User help

- **Ask:** type a question and press Enter. Choose *Deep* (plans, searches in parallel, verifies every claim) or *Fast* (single pass, quickest). Numbers in the answer are citations; select one to see its source. A claim drawn with a strike-through could not be matched to the sources. Conflicts and missing information are listed below the answer. Select *Trace* to see the plan, the searches and their timings.
- **Actions:** ask Cortex to draft or send something. It shows a card with the exact content; select *Edit* to change it, *Decline* to discard it or *Approve* to run it once. Nothing is sent without this step.
- **Brief:** open *Brief* for what changed in the last week and what is due soon, with sources; select *Ask* on a card to ask about it.
- **Memory:** say "remember that ..." in a question, or use *Teach Cortex something* on the Memory page. Each memory shows why it was kept; pin it to keep it from fading, edit it, or forget it.
- **Connections:** use the sample workspace to try Cortex without accounts, or connect your own apps with one click (when the server has a Composio key).
- **When something fails:** Cortex names the source that failed, why, and what may be missing from the answer, and continues with the others. If every model is busy, wait a minute or switch to Fast mode.

CHAPTER 12

CONCLUSION AND FUTURE SCOPE

Summary. In this project we designed and built Cortex, an agentic retrieval-augmented generation system that works over live work data. Its pipeline plans first and then executes, always within fixed limits. It splits each question into parts and searches several sources at once through one connector registry. The answer it streams must cite numbered evidence, and each claim is checked afterwards. Conflicts and missing information are reported. Actions are only ever proposed, never executed by the agent itself, and the memory it keeps is small and explains itself. Because a router moves between free-tier model providers, with failover, cooldowns and caching, the system costs nothing to run.

Conclusion. Over twelve questions the preliminary results back the main design choices. Mean groundedness was 0.88 and citation recall 0.92, the unanswerable question got an honest answer, and the median end-to-end time was 3.7 s. No action ran without consent in any run, and that does not depend on how the model behaves, because the design leaves no other path to execution. The evaluation also showed us where Cortex is weak. Conflict detection and action proposals each worked in only one of two cases, and the deadline questions had the lowest groundedness. Our base paper, GRASP, shows that both the granularity of evidence and the number of retrieval steps matter. Cortex follows that lesson by keeping evidence small and retrieval bounded, though with fixed rules where GRASP learns a policy.

Future scope.

- Make conflict detection independent of the synthesis model, for example by comparing dates and numbers extracted from evidence of different sources about the same entity.
- Enlarge the evaluation to the planned 50 questions, repeat runs to report variance, and add citation precision.
- Bring the Oracle AI Database backend into use for vector search over memories, and compare it with the SQLite keyword fallback.
- Implement the remaining vision items: time-travel queries over source snapshots, an editable plan before execution, and shareable reasoning receipts.
- Expose Cortex itself as an MCP server, so that other assistants can use its research and memory.
- Study a learned, granularity-aware retrieval policy in the spirit of GRASP once a trainable model and data are available.

- Deploy to a hosted environment with persistent storage for pending actions.

CHAPTER 13
REFERENCES

[1] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*. New York: McGraw-Hill Education, 8th ed., 2015.

[2] V. Gandhi, J. Lee, S. Todmal, F. Dernoncourt, R. Rossi, Z. Wang, and A. Lan, "GRASP: GRanularity-aware search policy for agentic RAG." arXiv preprint arXiv:2607.10463, 2026.

[3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.

[4] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *The Eleventh International Conference on Learning Representations (ICLR 2023)*, 2023.

[5] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, (Toronto, Canada), pp. 10014–10037, Association for Computational Linguistics, 2023.

[6] O. Press, M. Zhang, S. Min, L. Schmidt, N. A. Smith, and M. Lewis, "Measuring and narrowing the compositionality gap in language models," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, (Singapore), pp. 5687–5711, Association for Computational Linguistics, 2023.

[7] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024.

[8] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective retrieval augmented generation." arXiv preprint arXiv:2401.15884, 2024.

[9] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, "Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics:*

Human Language Technologies (Volume 1: Long Papers), Association for Computational Linguistics, 2024.

[10] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling large language models to generate text with citations," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, (Singapore), pp. 6465–6488, Association for Computational Linguistics, 2023.

[11] L. Gao, Z. Dai, P. Pasupat, A. Chen, A. T. Chaganty, Y. Fan, V. Zhao, N. Lao, H. Lee, D.-C. Juan, and K. Guu, "RARR: Researching and revising what language models say, using language models," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, (Toronto, Canada), pp. 16477–16508, Association for Computational Linguistics, 2023.

[12] S. Min, K. Krishna, X. Lyu, M. Lewis, W. tau Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, (Singapore), pp. 12076–12100, Association for Computational Linguistics, 2023.

[13] C. Packer, V. Fang, S. G. Patil, K. Lin, S. Wooders, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems." arXiv preprint arXiv:2310.08560, 2023.

[14] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23)*, (San Francisco, CA, USA), ACM, 2023.

[15] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.

[16] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model context protocol (MCP): Landscape, security threats, and future research directions." arXiv preprint arXiv:2503.23278, 2025.

[17] A. Singh, A. Ehtesham, S. Kumar, and T. T. Khoei, "Agentic retrieval-augmented generation: A survey on agentic RAG." arXiv preprint arXiv:2501.09136, 2025.

[18] S. Es, J. James, L. E. Anke, and S. Schockaert, "RAGAs: Automated evaluation of retrieval augmented generation," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, (St. Julian's, Malta), pp. 150–158, Association for Computational Linguistics, 2024.

[19] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[20] M. Chui, J. Manyika, J. Bughin, *et al.*, "The social economy: Unlocking value and productivity through social technologies." McKinsey Global Institute report, `https://www.mckinsey.com/industries/technology-media-and-telecommunications/our-insights/the-social-economy`, July 2012.

[21] M. Chui, E. Hazan, R. Roberts, A. Singla, K. Smaje, A. Sukharevsky, L. Yee, and R. Zemmel, "The economic potential of generative AI: The next productivity frontier," tech. rep., McKinsey & Company, June 2023.

[22] B. Jin, H. Zeng, Z. Yue, J. Yoon, S. Arik, D. Wang, H. Zamani, and J. Han, "Search-R1: Training LLMs to reason and leverage search engines with reinforcement learning." arXiv preprint arXiv:2503.09516, 2025.

[23] Anthropic, "Introducing the Model Context Protocol." `https://www.anthropic.com/news/model-context-protocol`, Nov. 2024. Accessed: Oct. 2026.

[24] P. Kulkarni, *Knowledge Innovation Strategy*. New Delhi: Bloomsbury Publishing India, 2015.

[25] E. Ries, *The Lean Startup: How Today's Entrepreneurs Use Continuous Innovation to Create Radically Successful Businesses*. New York: Crown Business, 2011.

[26] Department for Promotion of Industry and Internal Trade, Government of India, "Startup India." `https://www.startupindia.gov.in`. Accessed: Oct. 2026.

ANNEXURE A
REFERENCES

(Strictly in ACM Format)

The references cited in this report are listed below in the ACM reference format. The numbered list produced by BibTeX appears in the References chapter before the annexures.

1. Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*.

2. Michael Chui, James Manyika, Jacques Bughin, et al. 2012. *The Social Economy: Unlocking Value and Productivity through Social Technologies*. McKinsey Global Institute.

3. Michael Chui, Eric Hazan, Roger Roberts, et al. 2023. *The Economic Potential of Generative AI: The Next Productivity Frontier*. McKinsey & Company.

4. Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. 2024. RAGAs: Automated Evaluation of Retrieval Augmented Generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*. ACL, 150–158.

5. Varun Gandhi, Jaewook Lee, Shantanu Todmal, Franck Dernoncourt, Ryan Rossi, Zichao Wang, and Andrew Lan. 2026. GRASP: GRanularity-Aware Search Policy for Agentic RAG. arXiv:2607.10463.

6. Luyu Gao, Zhuyun Dai, Panupong Pasupat, Anthony Chen, Arun Tejasvi Chaganty, Yicheng Fan, Vincent Zhao, Ni Lao, Hongrae Lee, Da-Cheng Juan, and Kelvin Guu. 2023. RARR: Researching and Revising What Language Models Say, Using Language Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. ACL, 16477–16508.

7. Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. 2023. Enabling Large Language Models to Generate Text with Citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. ACL, 6465–6488.

8. Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. 2025. Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions. arXiv:2503.23278.

9. Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2024)*.

10. Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. 2025. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. arXiv:2503.09516.

11. Parag Kulkarni. 2015. *Knowledge Innovation Strategy.* Bloomsbury Publishing India.

12. Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020).*

13. Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173.

14. Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2023. FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing.* ACL, 12076–12100.

15. Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. 2023. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560.

16. Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST '23).* ACM. https://doi.org/10.1145/3586183.3606763

17. Roger S. Pressman and Bruce R. Maxim. 2015. *Software Engineering: A Practitioner's Approach* (8th ed.). McGraw-Hill Education.

18. Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A. Smith, and Mike Lewis. 2023. Measuring and Narrowing the Compositionality Gap in Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2023.* ACL, 5687–5711.

19. Eric Ries. 2011. *The Lean Startup.* Crown Business, New York.

20. Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use

Tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

21. Aditi Singh, Abul Ehtesham, Saket Kumar, and Tala Talaei Khoei. 2025. Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG. arXiv:2501.09136.

22. Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. ACL, 10014–10037.

23. Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. 2024. Corrective Retrieval Augmented Generation. arXiv:2401.15884.

24. Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations (ICLR 2023)*.

25. Anthropic. 2024. Introducing the Model Context Protocol. Retrieved from https://www.anthropic.com/news/model-context-protocol

26. Department for Promotion of Industry and Internal Trade, Government of India. Startup India. Retrieved from https://www.startupindia.gov.in

ANNEXURE B

LABORATORY ASSIGNMENTS ON PROJECT ANALYSIS OF ALGORITHMIC DESIGN

- To develop the problem under consideration and justify feasibilty using concepts of knowledge canvas and IDEA Matrix.
  Both tools come from Kulkarni [24]. The IDEA matrix (Table B.1) is a grid of verbs, from Increase, Drive, Educate and Accelerate down to Ignore, Decrease, Eliminate and Avoid, that sorts what a product should push and what it should drop. The knowledge canvas is a one-page view of the opportunity a product addresses. We use both to argue feasibility from the user's and the business side.

I	D	E	A
Increase	Drive	Educate	Accelerate
Improve	Deliver	Evaluate	Associate
Ignore	Decrease	Eliminate	Avoid

Table B.1: IDEA Matrix

  The IDEA matrix applied to Cortex is given in Table B.2, and the knowledge canvas in Table B.3.

- Project problem statement feasibility assessment using NP-Hard, NP-Complete or satisfy ability issues using modern algebra and/or relevant mathematical models.

  Evidence selection. Let $E = \{e_1, \ldots, e_N\}$ be the retrieved evidence, each with a value v_j (its usefulness for the question) and a cost w_j (its length in tokens), and let B be the prompt budget. Choosing the subset $S \subseteq E$ that maximises $\sum_{j \in S} v_j$ subject to $\sum_{j \in S} w_j \leq B$ is the 0/1 knapsack problem, whose decision version is NP-complete; if value is instead measured as the number of sub-questions covered, the problem generalises set cover and is NP-hard as well. Cortex does not solve these problems exactly. It bounds the input (at most 18 searches, at most 6 items per search, so $N \leq 108$), makes all costs nearly equal by compacting each quote, and then selects greedily by score, which takes $O(N \log N)$ time.

  Retrieval policy. Choosing which tool to call, with which query and at what granularity, over several turns, is a sequential decision problem under uncertainty; GRASP [2] learns such a policy with reinforcement learning. Cortex replaces the policy by a fixed plan of at most five sub-questions and one bounded retry, so the number of steps is constant.

Increase	Trust in answers: every factual sentence is cited and verified.
Improve	Coverage: all relevant sources are searched in parallel for each sub-question.
Ignore	Exhaustive ingestion of all user data; Cortex queries tools at question time instead.
Drive	Consent-first automation: drafts are prepared, the user decides.
Deliver	A cited answer within seconds (median 3.7 s in the evaluation).
Decrease	Time spent searching across tools and switching tabs.
Educate	Users about why each memory exists, where it came from and when it expires.
Evaluate	Every claim (groundedness) and every release (evaluation harness).
Eliminate	Silent resolution of contradictions and unrequested actions.
Accelerate	Repeated questions through response and search caches, and Fast Mode.
Associate	Evidence across sources, so that disagreements become visible risks.
Avoid	Paid APIs (free tiers only) and guessing (honest "not found").

Table B.2: IDEA matrix for Cortex

Opportunity	Knowledge workers spend a large share of their time looking for information [20]; generative AI targets this work [21].
Users	Developers, professionals and undergraduates using several work tools.
Knowledge sources	Gmail, Calendar, Drive, Notion, Slack, GitHub, MCP servers, the web; the user's own memories.
Value proposition	Cited, verified answers; visible conflicts; explainable memory; actions only with consent; zero cost.
Differentiation	Compared with typical RAG chat: claim-level verification, conflict reporting, honest gaps, consent gate, governed memory.
Feasibility (business)	No infrastructure cost on free tiers; open, modular design; risk from changing free-tier limits and third-party APIs (Section 5.3 and Table 5.2).

Table B.3: Knowledge canvas for Cortex

Complexity of one run. With $|C|$ sources, $p \leq 5$ sub-questions, N evidence items, $|A|$ answer tokens and a constant number of model calls (at most 5), the run takes $O(p\,|C| + N \log N + |L| \cdot (|c| + |E_c|))$ work besides the model and tool calls, i.e. polynomial time. The problem as implemented is therefore in class P, with model and tool calls treated as bounded oracle calls.

- input x,output y, y=f(x)

  $x = (q, m, H, u)$ and $y = f(x) = (A, L, G, K, R, W)$ with $f = f_{remember} \circ f_{propose} \circ f_{verify} \circ f_{synth} \circ f_{rank} \circ f_{search} \circ f_{plan}$ as defined in Section 1.9. The function f is not deterministic, because the model calls are not; the guarantees that hold for every output (each proposal is pending, each failed source is reported, at most five model calls) are enforced by code outside the model.

ANNEXURE C

LABORATORY ASSIGNMENTS ON PROJECT QUALITY AND RELIABILITY TESTING OF PROJECT DESIGN

It should include assignments such as

- Use of divide and conquer strategies to exploit distributed/parallel/ concurrent processing of the above to identify object, morphisms, overloading in functions (if any), and functional relations and any other dependencies (as per requirements). It can include Venn diagram, state diagram, function relations, i/o relations; use this to derive objects, morphism, overloading

  Divide and conquer. The question is divided into sub-questions; each sub-question is divided into one search per source; all searches are executed concurrently; the results are merged (deduplicated and ranked) and conquered by a single synthesis. Objects: Plan, SubQuestion, Evidence, Claim, Conflict, ActionProposal, MemoryItem. Morphisms: f_{search} maps a (source, query) pair to evidence, f_{verify} maps (answer, evidence) to claims. Overloading: `search`, `execute` and `status` are implemented differently by the five connector classes behind one interface. The state behaviour of a proposal is in Figure 6.6. Figure C.1 relates the sentences of an answer to the verification outcome.

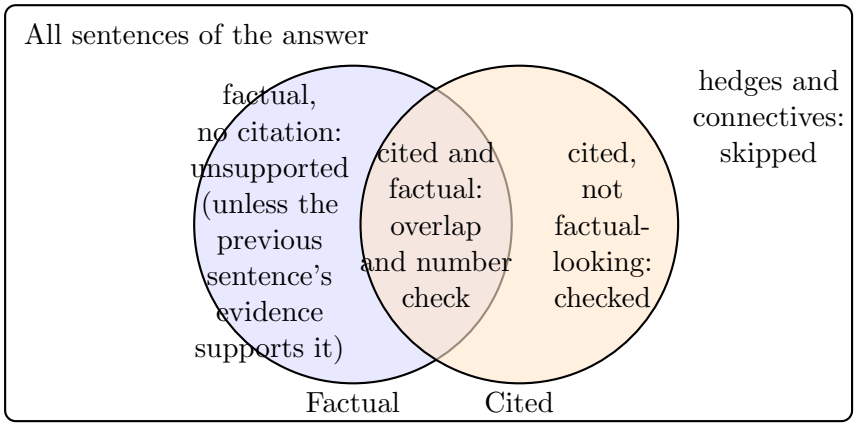


Figure C.1: Venn diagram of answer sentences and their verification

- Use of above to draw functional dependency graphs and relevant Software modeling methods, techniques including UML diagrams or other necessities using appropriate tools.

  The UML diagrams are given in the SRS and design chapters (use case, activity, state, class and sequence diagrams). Figure C.2 is the functional dependency graph of the main functions.

- Testing of project problem statement using generated test data (using mathematical models, GUI, Function testing principles, if any) selec-

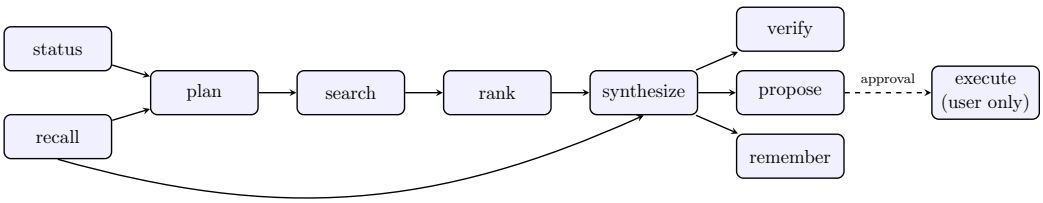


Figure C.2: Functional dependency graph

tion and appropriate use of testing tools, testing of UML diagram's reliability. Write also test cases [Black box testing] for each identified functions. You can use Mathematica or equivalent open source tool for generating test data.

Test data are generated by the sample workspace, whose dates are computed relative to the current day so that time-window questions always have data, and which contains deliberate cross-source conflicts; and by stub model providers and stub connectors in the automated tests. The tool used is Vitest. Black-box test cases for the identified functions are given in Table C.1; their results are covered by the tests in Chapter 9.

- Additional assignments by the guide. If project type as Entreprenaur, Refer [25],[21],[20], [26]

  This is an internal project, so the entrepreneurship assignments do not apply. The references are kept for completeness; the McKinsey reports are also used in the motivation. [Additional assignments by the guide, if any, to be added by the team.]

Function	Input	Expected output
plan (Fast)	“what changed this week”	One sub-question over all sources, window “last 7 days”
search	Source down (connection refused)	Failure with name, reason “could not be reached”, impact; run continues
rank	Duplicate items from two sub-questions	One copy kept; at most 16 (Deep) or 8 (Fast) items
synthesize	No evidence	“Not found” text, no model call
verify	“11 of 19 issues closed [2]” vs evidence “11 of 19”	supported
verify	“15 of 19 issues closed [2]” vs evidence “11 of 19”	partial (number mismatch)
verify	Factual sentence without citation	unsupported
propose	send_email without subject	Rejected: “missing subject”
execute	Approve by another user	Refused
execute	Second approve	Refused: “already executed”
remember	“Remember that X”	Memory X stored with salience 0.95

Table C.1: Black-box test cases for the identified functions

ANNEXURE D
PROJECT PLANNER

The project schedule is planned with a Gantt chart (Figure D.1) for the academic year 2026-2027. Bars show planned durations; the status column shows the state at the time of writing.

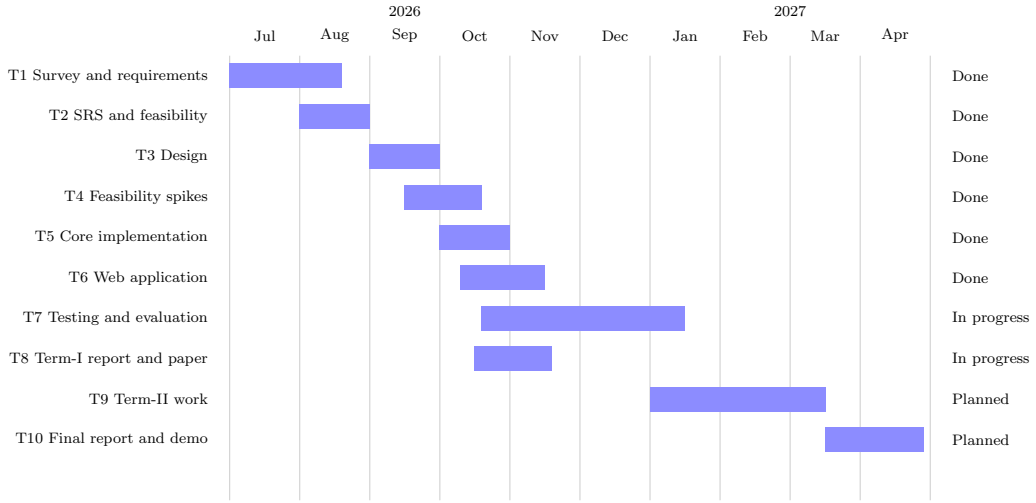


Figure D.1: Timeline chart (Gantt chart) of the project

Task	Description	Planned start	Planned end	Depends on
T1	Survey and requirements	Jul 2026	Aug 2026	–
T2	SRS and feasibility	Aug 2026	Aug 2026	T1
T3	Design	Sep 2026	Sep 2026	T2
T4	Feasibility spikes	Sep 2026	Oct 2026	T2
T5	Core implementation	Oct 2026	Oct 2026	T3, T4
T6	Web application	Oct 2026	Nov 2026	T5
T7	Testing and evaluation	Oct 2026	Jan 2027	T6
T8	Term-I report and paper	Oct 2026	Nov 2026	T3
T9	Term-II work	Jan 2027	Mar 2027	T7, T8
T10	Final report and demo	Mar 2027	Apr 2027	T9

Table D.1: Project planner

ANNEXURE E

REVIEWERS COMMENTS OF PAPER SUBMITTED

(At-least one technical paper must be submitted in Term-I on the project design in the conferences/workshops in IITs, Central Universities or UoP Conferences or equivalent International Conferences Sponsored by IEEE/ACM)

1. Paper Title: Agentic Retrieval-Augmented Generation over Live Workspace Data: A Survey and the Cortex Assistant (draft prepared in IEEE conference format; submission details to be filled by the team)
2. Name of the Conference/Journal where paper submitted : [to be filled by the team]
3. Paper accepted/rejected : [to be filled by the team]
4. Review comments by reviewer : [to be filled by the team]
5. Corrective actions if any : [to be filled by the team]

ANNEXURE F
PLAGIARISM REPORT

Plagiarism report: [to be attached by the team after the plagiarism check of the final report].

ANNEXURE G

TERM-II PROJECT LABORATORY ASSIGNMENTS

1. Review of design and necessary corrective actions taking into consideration the feedback report of Term I assessment, and other competitions/conferences participated like IIT, Central Universities, University Conferences or equivalent centers of excellence etc.
   Status: to be completed after the Term-I assessment. [Feedback and corrective actions to be added by the team.]

2. Project workstation selection, installations along with setup and installation report preparations.
   Status: done for development workstations; the setup is described in Chapter 11 and the hardware and software in Tables 4.2 and 8.1.

3. Programming of the project functions, interfaces and GUI (if any) as per 1 st Term term-work submission using corrective actions recommended in Term-I assessment of Term-work.
   Status: the functions, interfaces and GUI described in this report are implemented; Term-I corrective actions to be applied after the assessment.

4. Test tool selection and testing of various test cases for the project performed and generate various testing result charts, graphs etc. including reliability testing.
   Status: Vitest and the evaluation harness are in use (Chapter 9, Figures 10.8 and 10.9); a larger evaluation with repeated runs is planned.
   Additional assignments for the Entrepreneurship Project:

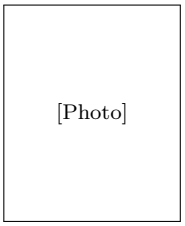
5. Installations and Reliability Testing Reports at the client end.
   Status: not applicable (internal project).

ANNEXURE H

INFORMATION OF PROJECT GROUP MEMBERS

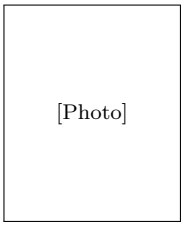
one page for each student .

1. Name : Ajit Paraskar



2. Date of Birth : [to be filled]
3. Gender : [to be filled]
4. Permanent Address : [to be filled]
5. E-Mail : [to be filled]
6. Mobile/Contact No. : [to be filled]
7. Placement Details : [to be filled]
8. Paper Published : [to be filled]

1. Name : Harshvardhan Patil



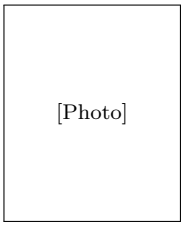
2. Date of Birth : [to be filled]
3. Gender : [to be filled]
4. Permanent Address : [to be filled]
5. E-Mail : [to be filled]
6. Mobile/Contact No. : [to be filled]
7. Placement Details : [to be filled]
8. Paper Published : [to be filled]

1. Name : Prathamesh Kumbhar

[Photo]

2. Date of Birth : [to be filled]
3. Gender : [to be filled]
4. Permanent Address : [to be filled]
5. E-Mail : [to be filled]
6. Mobile/Contact No. : [to be filled]
7. Placement Details : [to be filled]
8. Paper Published : [to be filled]

1. Name : Bikram Ghosh



2. Date of Birth : [to be filled]
3. Gender : [to be filled]
4. Permanent Address : [to be filled]
5. E-Mail : [to be filled]
6. Mobile/Contact No. : [to be filled]
7. Placement Details : [to be filled]
8. Paper Published : [to be filled]